Claude Certified Architect — Foundations Certification

مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر حقیقی **Claude Certified Architect — Foundations** فیصلہ کر سکتا ہے، Claude Code، Claude Agent SDK، Claude API، اور Model Context Protocol (MCP) یعنی — یہ امتحان trade-off حل نافذ کرتے وقت درست Claude-based دنیا کے بنیادی علم کا جائزہ لیتا ہے — کے ساتھ پروڈکشن ایپلیکیشنز بنائی جاتی ہیں Claude و بنیادی ٹیکنالوجیز جن سے

امتحانی سوالات حقیقت پسندانہ صنعتی منظرناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے agentic systems بنانا، multi-agent research pipelines ڈیزائن کرنا، Claude Code کو CI/CD میں ضم کرنا، developer productivity tools سے دستاویزات بنانا، اور غیر ساختہ structured data نکالنا

ہدف امیدوار

مثالی امیدوار ایک **solution architect** جو Claude اور پروڈکشن ایپلیکیشنز ڈیزائن کے ساتھ ship کرتا ہے آپ پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- **Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- **Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- **Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- **Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- **Context windows** — لمبی دستاویزات کے ساتھ کام، multi-agent context passing
- **CI/CD pipelines** — automated code review، test generation
- **Escalation and reliability** — error handling، human-in-the-loop

---

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (4 درست 1 میں سے)
اسکورنگ	100–1000 scale، passing score **720**
Guessing penalty	نہیں (ہر سوال کا جواب دیں!)
Scenarios	6 4 میں سے (randomly selected)

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	**27%**
2. Tool design and MCP integration	**18%**
3. Claude Code configuration and workflows	**20%**
4. Prompt engineering and structured output	**20%**
5. Context management and reliability	**15%**

امتحانی منظرنامہ

Scenario 1: Customer Support Agent

آپ ایک agent میں جو بناتے ہوئے Claude Agent SDK استعمال کرتے ہوئے returns، billing disputes، اور account issues کو سنبھالتا ہے agent MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) 80 ہدف کے ساتھ استعمال کرتا ہے+% first-contact resolution مناسب escalation کے ساتھ

Scenario 2: Claude Code کے ساتھ Code Generation

آپ Claude Code کو development میں استعمال کرتے ہیں تیز کرنے کے لیے: code generation، refactoring، debugging، documentation آپ کو اسے CLAUDE.md configuration اور custom slash commands کے ساتھ integrate کرنے کے سمجھنا ہے، اور کرنا planning mode کب استعمال کرنا ہے

Scenario 3: Multi-Agent Research System

ایک coordinator agent specialized subagents کو tasks کے سونپتا ہے: web research، document analysis، synthesis، اور report generation سسٹم کو citations کے ساتھ مکمل reports تیار کرنے چاہئیں

Scenario 4: Developer Productivity Tools

یہ agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور بنانے، routine tasks automate کرنے میں مدد دیتا ہے Built-in tools (Read، Write، Bash، Grep، Glob) اور MCP servers استعمال کیے جاتے ہیں

Scenario 5: Claude Code for Continuous Integration

Claude Code کو CI/CD pipeline میں ضم کریں تاکہ automated code reviews، test generation، اور pull request feedback ہو حاصل Prompts کے چاہئیں ہونے ڈیزائن ایسے false positives کم سے کم ہوں

Scenario 6: Structured Data Extraction

□□ معلومات نکالتا □□ دستاویزات سے □سسٹم غیر ساخت□، output کو JSON schemas □ذریعے □کو validate □□، کرتا
اور high accuracy □اسے □□□ برقرار رکھتا edge cases □□یے□□ کو درست طور پر سنبھالنا چا□

سرکاری دستاویزات

□وسیلہ	URL
Claude API — Messages	https://platform.claude.com/docs/en/api/messages
Claude API — Tool Use	https://platform.claude.com/docs/en/build-with-claude/tool-use
Claude API — Message Batches	https://platform.claude.com/docs/en/build-with-claude/message-batches
Claude Agent SDK — Overview	https://platform.claude.com/docs/en/agent-sdk/overview
Claude Agent SDK — Hooks	https://platform.claude.com/docs/en/agent-sdk/hooks
Claude Agent SDK — Subagents	https://platform.claude.com/docs/en/agent-sdk/subagents
Claude Agent SDK — Sessions	https://platform.claude.com/docs/en/agent-sdk/sessions
Model Context Protocol (MCP)	https://modelcontextprotocol.io/
MCP — Tools	https://modelcontextprotocol.io/docs/concepts/tools
MCP — Resources	https://modelcontextprotocol.io/docs/concepts/resources
MCP — Servers	https://modelcontextprotocol.io/docs/concepts/servers
Claude Code — Documentation	https://code.claude.com/docs/en/overview
Claude Code — CLAUDE.md and Memory	https://code.claude.com/docs/en/memory
Claude Code — Skills (incl. slash commands)	https://code.claude.com/docs/en/skills
Claude Code — Hooks	https://code.claude.com/docs/en/hooks
Claude Code — Sub-agents	https://code.claude.com/docs/en/sub-agents
Claude Code — MCP Integration	https://code.claude.com/docs/en/mcp
Claude Code — GitHub Actions CI/CD	https://code.claude.com/docs/en/github-actions
Claude Code — GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd
Claude Code — Headless (non-interactive mode)	https://code.claude.com/docs/en/headless

Prompt Engineering Guide	https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
Extended Thinking	https://platform.claude.com/docs/en/build-with-claude/extended-thinking
Anthropic Cookbook (code examples)	https://github.com/anthropics/anthropic-cookbook

---

□حصہ I: نظریاتی بنیادیں

یہ حصہ وہ تمام نظریہ احاطہ کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے۔ مواد کو مطابق — اس سے آپ کو domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts ٹیکنالوجیز اور موضوع کی زیادہ گہری سمجھ پیدا کر سکتے ہیں۔

---

1 باب: Claude API — Model Interaction کی بنیادیں

> دستاویزات: Messages API | Prompt Engineering

1.1 API Request Structure

Claude API ایک request–response model ہے۔ Claude Messages API کی پیروی کرتا ہے اور request میں یہ شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

اہم فیلڈز:

- `model` — model selection (`claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5`)
- `max_tokens` — response میں زیادہ سے زیادہ tokens کی تعداد
- `system` — system prompt (model کے رویے کی تعریف کرتا ہے)
- `messages` — conversation history (**تسلسل برقرار رکھنے کے لیے آپ کو مکمل history بھیجنی ہوتی ہے**)

- `tools` — دستیاب tools کی definitions
- `tool_choice` — tool selection strategy

1.2 Message Roles

`messages` array تین roles □□ استعمال کرتا:

- `user` — user messages
- `assistant` — model responses (history □یں □ جاتا □ وقت شامل کیا □ بھیجتا)
- `tool` — tool call results (role پر واضح طور پر set □ یا؛ □یں کیا جاتا `tool_result` content block کی صورت □یں ظا□ر □وتا □□)

ر□ انتہائی ا□م: Model requests □ کو بھیجنی چا□یا□ **conversation history** میں آپ کو **مکمل** API request آزاد □وتا □□□ call محفوظ □یں رکھتا — □ر state درمیان

1.3 Response میں `stop_reason` Field

Claude API response میں `stop_reason` □ کو □□ شامل □وتا □□، جو بتاتا □□ model □ generation کیوں روکی:

Value	Description	Action
`"end_turn"`	Model □ response مکمل کر لیا	□نتیجہ user کو دکھائیں
`"tool_use"`	Model tool call □□ کرنا چا□تا	Tool execute کریں اور result واپس دیں
`"max_tokens"`	Token limit ختم □و گئی	Response truncated □□ ممکن ;□□ limit □ بڑھانی پڑ
`"stop_sequence"`	Stop sequence مل گئی	اپنی application logic مطابق □ کو handle کریں

Agentic systems میں `"tool_use"` اور `"end_turn"` □ی — □یں ا□م □ سب سے □ agent loop کو کنٹرول کرتا □یں□

1.4 System Prompt

System prompt ایک خاص instruction جو □□ context اور behavioral rules □ یا □□□ متعین کرتا:

- `messages` array الگ □ یا؛ □وتا □یں □ کا حصہ □ `system` field □□ میں دیا جاتا
- user messages □□ پر فوقیت رکھتا
- conversation □□ میں لاگو ر□تا □□ □وتا اور پوری load ایک بار
- role، constraints، اور output format □□ استعمال □وتا □□ کی تعریف کو □ لیے

ا□م □ لیے □ امتحان: system prompt کی wording غیر ارادی tool associations طور □ مثال □ پیدا کر سکتی □□□ کو ضرورت سے □ زیاد□ `get_customer` کو model کی تصدیق کریں" جیسی □دایت customer پر، "□میشہ□ استعمال کرنے پر مجبور کر سکتی □□، حتیٰ کہ جب یہ ضروری نہ □و□

1.5 Context Window

Context window text (tokens) □جس □□ کی کل مقدار model ایک وقت میں process اس میں □□□ کر سکتا □□ شامل:

- System prompt
- مکمل message history
- Tool definitions
- Tool results

□م context-window مسائل:

1. **Lost-in-the-middle effect:** models □لمبا input □سے اعتماد زیادہ □□ معلومات کو آغاز اور اختتام پر موجود □کے کر سکتے □یں□ حل: □م معلومات شروع یا آخر □کے قریب کرتے □یں، مگر درمیان کی تفصیلات process □رکھیں
2. **Tool results □ر □ونا: کا جمع** tool call context میں output اگر □□□ شامل کرتا tool 40+ fields □کے واپس کرے □وں، تو □م 5 مگر صرف context □□□ کا بڑا حصہ ضائع □و جاتا
3. **Progressive summarization:** history compress وقت □کرتے numeric values، percentages، اور dates اکثر □("چند"، "کچھ"، "تقریباً") □یں □و جاتے □و کر غیر واضح □و کر گم

---

2 باب: Tools اور `tool_use`

> دستاویزات: Tool Use

2.1 `tool_use` □□ کیا

`tool_use` ایک mechanism جو □□ Claude کو external functions call □□□ کی اجازت دیتا □کرنے Model □□برا result کرتا □□ اور execute □اسے code بناتا □□؛ آپ کا structured tool call request □یں چلاتا — □و code راست واپس کرتا □□□

2.2 Tool Definition

□ر tool ایک JSON schema □کے ذریعے □کے define □□ کیا جاتا:

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile,
including name, email, order history, and account status. Use this tool BEFORE
lookup_order to verify the customer's identity. Accepts an email (format:
user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

Tool description لکھنے میں انتہائی اہم پہلو:

1. **Description tool selection کا بنیادی mechanism ہے۔** LLM tools کو ان کی descriptions کی بنیاد پر منتخب کرتا ہے۔ Minimal descriptions ("Customer information retrieve کرتا ہے") اُن مواقع پر غلطیوں کا باعث بنتی ہیں جب tools سے دوسرے ایک overlap کرتے ہوں۔
2. **Description میں شامل کریں:**
   - Tool کیا کرتا ہے اور کیا واپس کرتا ہے
   - Input formats اور example values
   - Edge cases اور constraints
   - کب مقابلے میں کب استعمال کرنا ہے tool similar alternatives کو
3. **یا ایک جیسے overlapping descriptions** اگر **بچیں** سے `analyze_content` اور `analyze_document` کی انہیں خلط ملط کر دے گا model تقریباً ایک جیسی ہوں، تو descriptions
4. **Built-in tools بمقابلہ MCP tools:** agents similar functionality والے built-in tools (Read، Grep) کو MCP tools پر ترجیح دے سکتے ہیں۔ اس سے بچنے کے لیے MCP tool descriptions مضبوط بنائیں — concrete advantages، unique data، یا وہ context نمایاں کریں جو built-in tools فراہم نہیں کر سکتے۔

2.3 `tool_choice` Parameter

`tool_choice` کے ذریعے کنٹرول کرتا ہے model tools کیسے منتخب کرتا ہے:

Value	Behavior	کب استعمال کریں
`{ "type": "auto" }`	Model کے خود طے کرتا ہے tool call کرنا یا text میں جواب دینا	default زیادہ تر حالات میں
`{ "type": "any" }`	Model کو لازماً کوئی tool call کرنا ہوگی	جب آپ کو guaranteed structured output چاہیے
`{ "type": "tool", "name": "extract_metadata" }`	Model کو لازماً ایک مخصوص tool call کرنی ہوگی	جب آپ کو forced first step / execution order چاہیے

□م منظرنامہ:

- `tool_choice: "any"` + multiple extraction tools → model □ترین tool منتخب کرتا □□، مگر پھر بھی structured output □□ ملتا
- Forced selection → جب آپ کو کسی خاص پ□لی action □یہ کی ضمانت چا□ئے (مثلاً enrichment □□ پ□لا □سے `extract_metadata`)

2.4 Structured Output □لیے □کے JSON Schemas

حاصل کرنے کا **سب سے قابلِ** structured output □سے Claude استعمال کرنا `tool_use` □کے ساتھ JSON schemas □یہ □□□ طریقے **اعتماد**:

- Syntax درست □اعتبار سے □کے JSON □□ کی ضمانت دیتا (braces □وتے □یں غائب نہ□یں، trailing commas نہ□یں □وتے)
- □مطلوبہ structure □□ نافذ کرتا (required fields □یں □وتے □وجود)
- Semantic correctness کی ضمانت نہ□یں دیتا (values □یں □و سکتی □و غلط پھر بھی)

Schema design — بنیادی اصول:

```
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "Details if category = 'other' or 'unclear'"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "Null if the information was not found in the source"
    }
  },
  "required": ["category", "severity"]
}
```

Schema design rules:

1. **Required □بمقابلہ optional:** بنائیں جن کی معلومات □میشہ □دستیاب □و□ fields کو required □ہ صرف اُن گھڑنے □پر مجبور کر سکتی □یں values غائب □ونے □پر data کو Required fields model

2. **Nullable fields:** ایسی معلومات کے لیے `"type": ["string", "null"]` استعمال کریں جو ممکنہ طور پر موجود نہ ہوں Model `null` خود ساختہ وجہ کے بجائے اس کے واپس کر سکتا ہے value بنائے
3. **Enums with `"other"`:** اپنی predefined categories کے لیے سے باہر کی معلومات ضائع ہونے سے بچانے کے لیے `"other"` + ایک detail string شامل کریں
4. **Enum `"unclear"`:** جب کے لیے اُن صورتوں کے لیے model category ایماندار — ہو نہ ہو کے بارے میں پُراعتماد نہ بننے سے بہتر ہے غلط category `"unclear"`

2.5 Syntax بمقابلہ Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON، غلط field type	JSON schema کے ساتھ `tool_use` (ختم کر دیتا ہے)
Semantic	Totals match کرتے نہیں، value غلط field میں، hallucination	Validation checks، feedback کے ساتھ retry، self-correction

---

باب 3: Claude Agent SDK — Agentic Systems بنانا

> دستاویزات: Agent SDK | Hooks | Subagents | Sessions

3.1 Agentic Loop کیا ہے

Agentic loop خودکار task execution کا بنیادی pattern ہے۔ Model صرف جواب نہیں دیتا — وہ actions کی ایک sequence انجام دیتا ہے:

```
1. Claude کو tools کے ساتھ request بھیجیں
2. Response وصول کریں
3. stop_reason چیک کریں:
   - "tool_use" -> tool execute کریں، result history میں append کریں، step 1 پر واپس جائیں
   - "end_turn" -> task مکمل، result user کو دکھائیں
4. مکمل ہونے تک repeat کریں
```

یہ model-driven approach ہے: Claude اپنے context اور پچھلے tool results کی بنیاد پر اگلا tool خود منتخب کرتا ہے۔ یہ fixed ہوتی ہے action sequence سے مختلف ہے جہاں hard-coded decision trees

Anti-patterns (ان سے بچیں):

- Completion detect کرنے کے لیے assistant text parse کرنا ("Task completed")
- Primary stop condition کے طور پر arbitrary iteration limit استعمال کرنا (مثلاً `max_iterations=5`)
- سمجھنا completion signal دیا ہے یا نہیں، اسے textual content سے assistant نہ چیک کرنا

درست طریقہ: completion کا واحد قابلِ اعتماد signal `stop_reason == "end_turn"` ہے

3.2 `AgentDefinition` Configuration

`AgentDefinition` Claude Agent SDK میں agent configuration object □□:

```
agent = AgentDefinition(
    name="customer_support",
    description="Handles customer requests for returns and order issues",
    system_prompt="You are a customer support agent...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
    # Coordinator میں کہ:
    # allowed_tools=["Task", "get_customer", ...]
)
```

م□l parameters:

- `name` / `description` — agent کی شناخت اور وضاحت
- `system_prompt` — ہدایات والا □ system prompt
- `allowed_tools` — □ اجازت یافتہ tools کی فہرست (least privilege اصول)

3.3 Hub-and-Spoke: Coordinator اور Subagents

Multi-agent architecture عموماً hub-and-spoke topology □□ میں بنائی جاتی:

```
        Coordinator
       /     |      \
  Subagent1  Subagent2  Subagent3
   (search)   (analysis)   (synthesis)
```

Coordinator کی ذمہ □ داریاں:

- Task کو subtasks میں تقسیم کرنا
- □ سہ کون □ طہ کرنا کہ □ یہ subagents میں □ درکار (dynamic selection)
- subagents کو سونپنا کام
- نتائج جمع اور validate کرنا
- Errors اور retries handle کرنا
- نتائج user تک پہ□نچانا

م□l اصول: subagents کا context □□□ الگ □وتا

- Subagents خود بخود coordinator کی conversation history inherit □نہیں کرتے
- تمام ضروری context subagent prompt □□ میں **واضح طور پر** دینا □وتا
- Subagents calls درمیان □ کہ memory share □نہیں کرتے
- تمام communication coordinator □□ گزرتی □ ذریعہ □ کہ (observability اور error control □ لیے □ کہ)

3.4 Subagents □لیے □کے بنانے □ `Task` Tool

Subagents `Task` tool □ذریعے □کے spawn □یں □جاتے □کیے:

```
# Coordinator کے allowedTools میں "Task" ہونا چاہیے
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

Explicit context passing □□ لازمی:

```
# Bad: subagent پاس کے context نہیں ہے
Task: "Analyze the document"

# Good: مکمل context prompt میں
Task: "Analyze the following document.
Document: [full document text]
Prior search results: [web search results]
Output format requirements: [schema]"
```

Parallel spawning: coordinator ایک response میں متعدد `Task`s call □□ کر سکتا — subagents parallel □یں □چلتے:

```
# Coordinator ایک کے response ہے شامل ہے یہ:
Task 1: "Search for articles about X"
Task 2: "Analyze document Y"
Task 3: "Search for articles about Z"
# تینوں ایک ساتھ چلتے ہیں
```

3.5 Agent SDK میں Hooks

Hooks agent lifecycle مخصوص □کے points پر interception اور transformation □یں □کی اجازت دیتے □یں

PostToolUse tool result کو model □تک پہنچانے □سے پہلے intercept □□ کرتا:

```
# مثال: MCP tools کے تاریخ کے formats normalize کریں
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Unix timestamp -> ISO 8601
    # "Mar 5, 2025" -> "2025-03-05"
    return normalized_result
```

Outgoing-call interception hook policy □والے □کی خلاف ورزی کرنے actions کو block □□ کرتا:

```
# مثال: $500 سے اوپر refunds block کریں
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

Hooks اور prompt instructions میں فرق

Attribute	Hooks	Prompt instructions
Guarantee	**Deterministic** (100%)	**Probabilistic** (>90%, 100% نہیں)
کب استعمال کریں	Critical business rules، financial operations، compliance	General preferences، recommendations، formatting
مثال	Refunds > $500 block کریں	“Try to solve before escalating”

قاعدہ: جب failure ہوں کہ مالی، قانونی، یا حفاظتی نتائج — prompts نہیں، hooks استعمال کریں

باب 4: Model Context Protocol (MCP)

> دستاویزات: MCP | Tools | Resources | Servers

4.1 MCP کیا ہے

Model Context Protocol (MCP) ایک open protocol جو ہے external systems کو Claude سے جوڑتا ہے MCP تین بنیادی resource types define کرتا ہے:

1. **Tools** — functions جنہیں agent actions لیے کے لیے انجام دینے call کر سکتا ہے (CRUD operations، API calls، command execution)
2. **Resources** — context لیے کے data جسے agent پڑھ سکتا ہے (documentation، database schemas، content catalogs)
3. **Prompts** — عام tasks لیے کے predefined prompt templates

4.2 MCP Servers

MCP server ایک process جو ہے MCP protocol implement اور کرتا ہے tools/resources جب فراہم کرتا ہے آپ MCP server سے connect ہیں کرتے:

- تمام tools خود بخود discover ہیں ہو جاتے ہیں
- تمام connected servers کے tools میں ہوتے ہیں ایک ساتھ دستیاب
- Tool descriptions کے ہیں کرتی طے model گا کرے استعمال انہیں کیسے

4.3 MCP Servers کو Configure کرنا

Project configuration (`.mcp.json`) — team usage □لیے □کے:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

□م نکات:

- `.mcp.json` project root اور □□ □وتا میں محفوظ version control □□ □وتا میں
- Environment variables (`${GITHUB_TOKEN}`) secrets □یں □وتے □استعمال □لیے □کے — tokens خود commit □نہیں کیے جاتے
- □یہ project تمام □کے contributors □□ □وتا دستیاب □لیے □کے

User configuration (`~/.claude.json`) — personal/experimental servers □لیے □کے:

- User □کے home directory □□ □وتا میں محفوظ
- Version control □ذریعے □کے share □وتا □نہیں
- Personal experiments اور testing موزوں □لیے □کے

Servers کا انتخاب:

- Standard integrations (Jira، GitHub، Slack) موجود □سے □□□ پہلے □لیے □کے community MCP servers کو ترجیح دیں
- اپنی custom servers صرف unique، team-specific workflows بنائیں □لیے □کے

4.4 MCP میں `isError` Flag

جب MCP tool میں error تو □□ آتا response میں `isError: true` □یہ □□□ □وتا استعمال agent □کو بتاتا □□ کہ call fail □□□ □وئی

Structured error (اچھا):

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable. Timeout while calling the
orders API.",
    "attempted_query": "order_id=12345",
    "partial_results": null
  }
}
```

Generic error (anti-pattern):

```
{
  "isError": true,
  "content": "Operation failed"
}
```

Generic error agent کیا — کوئی معلومات نہیں دیتا کہ لیے فیصلہ سازی کو retry کریں، query بدلیں، یا escalate کریں؟

4.5 MCP Resources

Resources ایسا data جو ہیں agent context کے لیے حاصل کرنے کے actions ہیں جو بغیر طلب کر سکتا ہے:

- Content catalogs (مثلاً تمام project tasks کی فہرست، hierarchical navigation)
- Database schemas (data structure کے لیے سمجھنے)
- Documentation (API references، internal guides)
- Issue/task summaries

Resource کا فائدہ: agent کے لیے سمجھنے کو یہ exploratory tool calls کے پڑنے کی ضرورت نہیں کہ data موجود ہے کیا Resource فوری “map” ہے فراہم کرتا ہے

باب 5: Claude Code — Configuration اور Workflows

> دستاویزات: Claude Code | Memory / CLAUDE.md | Skills | MCP | Hooks | Sub-agents | GitHub Actions | Headless

5.1 CLAUDE.md Hierarchy

CLAUDE.md □و instruction file(s) جو □یں Claude Code سطحی تین کی اس □□□ استعمال کرتا hierarchy □□:

```
1. User-level: ~/.claude/CLAUDE.md
   - صرف اسی user پر لاگو
   - VCS کہ ذریعہ share ہوتا نہیں
   - ذاتی preferences اور working style

2. Project-level: .claude/CLAUDE.md یا root CLAUDE.md
   - تمام project contributors پر لاگو
   - VCS کہ ذریعہ manage □□ ہوتا
   - Coding standards، testing standards، architectural decisions

3. Directory-level: subdirectories میں CLAUDE.md
   - جب اسی directory کی files □□ ہوتا لاگو ہو تو کہ ساتھ کام
   - codebase مخصوص کہ اُس حصہ کہ conventions
```

□ایک نئ **عام غلطی:** team member کو project instructions □و □نہیں ملتیں کیونکہ `.claude/CLAUDE.md` (project-level) □کہ بجائ `~/.claude/CLAUDE.md` (user-level) □میں رکھی گئی تھیں

5.2 `@path` Syntax (File Imports)

CLAUDE.md بیرونی files کو `@path` □ذریعہ □کہ refer □جس سہ، □□ کر سکتا configuration modular جاتی □و □□:

```
# Project CLAUDE.md

Coding standards @./standards/coding-style.md میں بیان کیہ گئہ میں
Test requirements @./standards/testing-requirements.md میں میں
Project overview @README.md اور میں dependencies @package.json میں میں
```

`@path` قواعد □کہ:

- `@` فوراً بعد □کہ file path □آئ (کوئی space □نہیں)
- Relative اور absolute دونوں paths supported □یں
- Relative paths اُس file مطابق □کہ resolve میں جس □یں □وتہ □ import □□
- Maximum import nesting depth 5 □□

□اس سہ duplication □ر اور □□ کم □وتی package □متعلقہ میں صرف standards □یں □وتہ □ شامل

5.3 `.claude/rules/` Directory

`.claude/rules/` monolithic CLAUDE.md جو، □□ متبادل کا rules کو topic □حساب سہ □کہ organize □کہ □کرنہ □وتا استعمال □لیہ □□:

```
.claude/rules/
  testing.md          -- testing conventions
  api-conventions.md  -- API conventions
  deployment.md       -- deployment rules
  react-patterns.md   -- React patterns
```

YAML frontmatter ساتھ □کا `paths` conditional loading:

```
---
paths: ["src/api/**/*"]
---

API files □کے لیے □کے ساتھ explicit error handling □استعمال کریں
□ہر endpoint standard response wrapper □واپس کرے
```

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---

Tests میں describe/it blocks □استعمال □ہونے چاہئیں
Data factories □استعمال کریں، hardcoding □نہیں
Database mock □نہ کریں — test database □استعمال کریں
```

□□ کیسے □کام کرتا □ہے:

- Rule صرف اس وقت load جب □□ □ہوتا Claude Code ایسا file edit جو □کرے `paths` pattern □سے match کرتا □ہو
- □اس context اور tokens □غیر متعلق — □میں □بچتے rules load □ہوتے □نہیں
- Glob patterns آپ کو file type □کے لحاظ سے □کے conventions apply □□چاہے، □میں □دیتے □کرنے files codebase میں □ہوں بھی □کہیں (tests مفید □ہت □لیے □کے)

`paths` □والے `.claude/rules/` □بمقابلہ directory-level CLAUDE.md کب استعمال کریں:

- `.claude/rules/` with `paths` — جب conventions کئی directories میں بکھری files (tests، migrations) پر لاگو □ہوں
- Directory-level CLAUDE.md — جب conventions کسی خاص directory اور □کہیں اور □ہوں وابستہ □سے ضروری نہ □ہوں

5.4 Custom Slash Commands اور Skills

> **نوٹ:** □موجود Claude Code version میں custom commands (`.claude/commands/`) کو skills (`.claude/skills/`) ساتھ □کا unify دونوں □□□ کر دیا گیا formats `/name` commands □بناتے □□□ supported اب بھی format کا ذکر کرتی □□ — □وہ `.claude/commands/` □میں □امتحانی گائیڈ

Slash commands reusable prompt templates جو □میں `/name` □ذریعے □کا invoke □میں □ہوتے □ہیں:

`.claude/commands/` format (legacy، supported):

```
.claude/commands/
  review.md        -- /review -- standard code review
  test-gen.md      -- /test-gen -- test generation
```

`.claude/skills/` **format (current):**

```
.claude/skills/
  review/SKILL.md  -- /review -- frontmatter configuration کے ساتھ
  test-gen/SKILL.md
```

Project commands (`.claude/commands/` یا `.claude/skills/`):

- VCS اور میں محفوظ repo clone میں دستیاب ہوتے ہیں کرنے والے سب کے لیے
- ٹیم میں consistent workflows یقینی بناتے ہیں

User commands (`~/.claude/commands/` یا `~/.claude/skills/`):

- ذاتی commands جو VCS کے ذریعے share نہیں ہوتے
- انفرادی workflows کے لیے

5.5 Skills — `.claude/skills/`

Skills advanced commands ہیں جو SKILL.md frontmatter کے ذریعے configure ہوتے ہیں:

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "Path to the directory to analyze"
---

دی گئی directory میں code structure کا تجزیہ کریں۔
Dependencies اور architectural patterns پر ایک report تیار کریں۔
```

Frontmatter parameters:

Parameter	Description
`context: fork`	Skill کو isolated subagent میں چلاتا ہے۔ Verbose output main session کو pollute نہیں کرتا
`allowed-tools`	یہ tools محدود کرتا ہے کہ کون سے دستیاب ہوں گے (security — مثلاً skill files delete نہیں کر سکے گی اگر اجازت نہ ہو)
`argument-hint`	جب skill parameters کے بغیر invoke ہو تو argument کا اشارہ مانگنے

Skill بمقابلہ CLAUDE.md کب استعمال کریں:

- **Skill** — مخصوص task کے لیے on-demand invocation (review، analysis، generation)
- **CLAUDE.md** — ہمیشہ لوڈ ہونے والے general standards اور conventions

Personal skills (`~/.claude/skills/`):

- اپنی personal variants □کو بنائیں □سے مختلف ناموں teammates متاثر نہ □ہوں

5.6 Planning Mode □بمقابلہ Direct Execution

Planning mode:

- Model صرف investigate اور plan □□؛ کرتا changes نہیں کرتا
- Codebase explore □کرنے کے لیے Read، Grep، Glob □□ استعمال کرتا
- ایک implementation plan □جسے □□ تیار کرتا user approve □□ کرتا
- بغیر side effects □کے safe exploration

Planning mode کب استعمال کریں:

- □بڑے changes (درجنوں files)
- Multiple plausible approaches (microservices: boundaries □کیسے define □ہوں؟)
- Architectural decisions (کون سا framework؟ structure کیا؟)
- Unfamiliar codebase (change □□ سمجھنا ضروری □ہو□ پہلے □سے کرنے)
- Library migrations 45 جو+ files کو متاثر کریں

Direct execution کب استعمال کریں:

- Single-file fixes واضح stack trace ساتھ □کے
- ایک validation check شامل کرنا
- changes اچھی طرح سمجھ□ گئے□، غیر مبہم

Combined approach:

1. Investigation اور design □کے لیے planning mode
2. User plan approve □کریں
3. Approved plan implement □کرنے کے لیے direct execution

Explore subagent — codebase explore □کرنے کے لیے specialized subagent:

- Verbose output کو main context □سے isolate □□ کرتا
- صرف summary □□ واپس کرتا
- Multi-phase tasks میں context-window exhaustion □□ روکتا

5.7 `/compact` Command

`/compact` context compress □کرنے کے لیے built-in command □□:

- Pچھلی history کو summarize □کر کے context window □□ میں جگہ خالی کرتا
- Long investigation sessions جب □□ہوتا استعمال میں verbose tool output □سے context بھر جائے
- Risk: exact numeric values، dates، اور specific details summarization میں □ہو سکتے □ہیں گم

5.8 `/memory` Command

`/memory` sessions درمیان □ memory manage □لیے □ کرنے □ built-in command □□:

- `CLAUDE.md` file editing □تاکہ، □□ کھولتا □لیے □ notes، preferences، اور context محفوظ کیے جا سکیں
- Information sessions اور □□ برقرار رہتی □ پار startup پر خود بخود load □□ ہوتی□
- Project conventions، user preferences، frequently used commands، اور current work context محفوظ کرنے □ کے لیے مفید
- ہر session میں وہی instructions کا متبادل دوبارہ سمجھانے □

5.9 Claude Code CLI for CI/CD

`-p` (یا `--print`) flag:

```
claude -p "Analyze this pull request for security issues"
```

- Non-interactive mode: prompt process □□ کرتا، stdout پر print اور، □□ کرتا exit □□ ہو جاتا□
- User input کا انتظار نہیں کرتا
- CI/CD pipelines میں Claude □چلانے □ کا واحد درست طریقہ

CI □لیے □ structured output:

```
claude -p "Review this PR" --output-format json --json-schema
'{"type":"object",...}'
```

- `--output-format json` — output کو JSON □□ میں دیتا
- `--json-schema` — output کو schema مطابق □ validate □□ کرتا
- Result کو automatically inline PR comments □لیے □ پوسٹ کرنے □ parse □□ کیا جا سکتا

Session context isolation: میں کم مؤثر review کیا □ہو، اکثر code generate جس نہ Claude session □ہی □ کرنے □ کا امکان کم □ہوتا challenge ساتھ رکھتا □□ اور اپنے □ فیصلوں کو reasoning context اپنی (model □ہوتی □□)□ Review □لیے □ independent instance استعمال کریں

Duplicate comments □ئے: بچاؤ: □سے commits □ بعد دوبارہ □ review □پچھلے، وقت □کرتے review results context □کرنے □ کی □ہدایت دیں□ unresolved issues report کو صرف نئے □ یا Claude میں شامل کریں اور

5.10 `fork_session` اور Session Management

`--resume <session-name>` named session کو resume □□ کرتا:

```
claude --resume investigation-auth-bug
```

- saved context ساتھ پچھلی □ conversation □□ جاری رکھتا

- Long investigations جو متعدد sessions مفید □ل□ ان ک□ پر پھیلی □وں
- Risk: اگر پچھلی session بعد □ک files تو □وں بدل چکی tool results stale □یں □و سکت□

`fork_session` shared context □س independent branch □□ بناتا:

```
Codebase investigation
         |
    fork_session
    /           \
Approach A:      Approach B:
Redux            Context API
```

- دونوں forks branch point تک context inherit □یں □کرت□
- آزادان□ طور پر □و بعد اس ک□ diverge □یں □کرت□
- Approaches compare □کرن□ یا strategies test مفید □ل□ ک□ کرن□

Resume نئی □بجائ□ ک□ session کب شروع کریں:

- Tool results stale □وں (files □وں بدل چکی)
- کافی وقت گزر چکا □و اور context degrade □و گیا □و
- □پران tool data ساتھ □ک resume: □□ □ک □ک "□م ن□ جو پایا اس کا مختصر خلاص□ □ی □□ بہتر □□ بجائ□ □ی کرن□ □س "... restart □کیا جائ□

---

6 باب: Prompt Engineering — Advanced Techniques

> دستاویزات: Prompt Engineering | Anthropic Cookbook

6.1 Few-shot Prompting

Few-shot prompting جس میں □□ □طریق□ □و expected behavior □ل□ ک□ دکھان□ prompt 4–2 میں input/output examples □یں□ □جات□ شامل کی□

Few-shot text descriptions □□ مؤثر کیوں □زیاد□ □س:

- □م□ instruction □زیاد“ □جیس□ precise □□ سمجھا جا سکتا □س طریقوں کو کئی ”□وں
- Example غیر مب□م طور پر expected format اور decision logic □□ دکھاتا
- Model pattern □نئ□ کو cases پر generalize □□ کرتا (صرف examples □یں□راتا ن□)

Few-shot examples کی اقسام اور استعمال:

1. **Ambiguous scenarios □ل□ ک□ examples:**

```
Request: "My order is broken"
Action: Call get_customer -> lookup_order -> check status.
Rationale: “broken” damaged item کو آپ ؛ سکتا ہو بھی order details درکار میں

Request: "Get me a manager"
Action: فوراً escalate_to_human call کریں.
Rationale: Customer واضح طور پر human کی کوشش نہ کریں خود حل کرنے مانگ رہا
```

2. **Output formatting کے لیے examples:**

```
Finding example:
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

3. **Acceptable بمقابلہ problematic code کے لیے الگ کرنے examples:**

```
// Acceptable (flag نہ کریں):
const items = data.filter(x => x.active);

// Problem (flag کریں):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

4. **Different document formats سے extraction کے examples:**

```
Document with inline citations:
"As shown in the study (Smith, 2023), the rate is 42%."
-> {"value": "42%", "source": "Smith, 2023", "type": "inline_citation"}

Document with bibliography references:
"The rate is 42%. [1]"
-> {"value": "42%", "source": "reference_1", "type": "bibliography"}
```

5. **Informal measurements کے لیے examples:**

```
Text: "about two handfuls of rice"
-> {"amount": "~100g", "original_text": "two handfuls", "precision": "approximate"}

Text: "a pinch of salt"
-> {"amount": "~1g", "original_text": "a pinch", "precision": "approximate"}
```

Few-shot خاص طور پر informal اور non-standard measurement units جو میں مؤثر نکالنے purely rule-based instructions سے بہتر متنوع ہوتی ہیں کے لیے

Prompts میں format normalization rules: جب structured output کے لیے strict JSON schemas استعمال کریں، prompt میں normalization rules شامل کریں:

```
Normalization:
- Dates: ہمیشہ ISO 8601 (YYYY-MM-DD); "yesterday" -> absolute date compute کریں
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency":
"USD"}
- Percentages: decimal fraction; "half" -> 0.5
```

□اس سے semantic errors ہاں جہ□یں □وتے □ کم JSON syntactically ہو مگر درست □ہو values inconsistent □ہوں

6.2 Explicit Criteria □بمقابلہ Vague Instructions

Bad (vague):

```
Code comments کی accuracy چیک کریں۔
Conservative صرف — رہیں high-confidence findings report کریں۔
```

Good (explicit criteria):

```
Comment کو problematic صرف اُس صورت میں flag کریں جب:
1. Comment کا بیان actual code behavior سے CONTRADICT ہو کرتا
2. Comment کسی غیر موجود function یا variable کا حوالہ دے
3. TODO/FIXME comment کسی ایسے bug جو کرے اشارہ کی طرف code کی پہلے ہی fix ہو چکا ہو میں

Flag نہ کریں:
- ایسے comments جو صرف style کے لحاظ سے پرانے ہوں
- معمولی wording inaccuracies
- Missing comments (یہ الگ category ہے)
```

Severity criteria examples □ساتھ کے define کریں:

```
CRITICAL: User کے لیے runtime failure
  Example: Payment process کرتے وقت NullPointerException

HIGH: Security vulnerability
  Example: SQL injection، XSS، missing authorization checks

MEDIUM: Logic bug بغیر فوری impact کے
  Example: Wrong sorting، off-by-one error

LOW: Code quality
  Example: Duplication، چھوٹے data کے لیے suboptimal algorithm
```

6.3 Prompt Chaining

Prompt chaining ایک complex task کو focused steps کی sequence □□ میں توڑتا:

```
Step 1: auth.ts analyze کریں (صرف local issues)
       -> Output: auth.ts میں issues کی list

Step 2: database.ts analyze کریں (صرف local issues)
       -> Output: database.ts میں issues کی list

Step 3: Integration pass (cross-file dependencies)
       -> Output: module boundaries پر issues
```

□□ کیوں اہم □ہے:

- **Attention dilution** جب—□□ بچاتا □سے model سی بہت ایک ساتھ files تو کچھ □□ دیکھتا files □کے bugs miss □د سکتا □□ shallow commentary کر سکتا □□ اور باقی پر
- ہر file □لیے □کے consistent analysis quality □□ یقینی بناتا
- Cross-file interactions الگ □لیے □کے analysis □□ کی اجازت دیتا

Prompt chaining □بمقابلہ dynamic decomposition کب استعمال کریں:

- **Prompt chaining** — predictable، repeatable tasks (code review، file migrations)
- **Dynamic decomposition** — open-ended investigations جہاں subtasks execution □ہوتے □کے دوران واضح □ہیں

6.4 “Interview” Pattern

Solution implement □کرنے □سے پہلے Claude clarifying questions □□ پوچھتا:

```
Claude: "API کے لیے caching implement کرنے سے پہلے چند سوالات:
1. آپ TTL یا event-based میں سے کون سی cache invalidation strategy پسند کرتے ہیں؟
2. اگر cache unavailable ہو تو کیا stale data قابلِ قبول ہے؟
3. Caching per-user ہونی چاہیے یا global؟
4. Cache کرنے کے لیے expected data volume کیا ہے؟"
```

□□ کب مفید □ہے:

- Unfamiliar domain (fintech، healthcare، legal systems)
- غیر واضح implications □والے tasks (cache strategies، failure modes)
- متعدد viable approaches انتخاب □جہاں بہترین context □پر منحصر

6.5 Validation اور Retry-with-Feedback

جب extracted data validation fail □کرے:

```
Step 1: Document سے data extract کریں
Step 2: Validate کریں (Pydantic، JSON Schema، business rules)
Step 3: اگر error ہو — context کے ساتھ retry کریں:
  - Original document
  - Previous (incorrect) extraction
  - Specific error: "Field 'total' = 150, but sum(line_items) = 145. Re-check values."
```

Retry □کب مؤثر □ہوگا:

- Format errors (date غلط format میں)
- Structural errors (field □غلط جگ□)
- Arithmetic inconsistencies (model □دوبار check □□ کر سکتا)

Retry گا □کر مدد نہ□یں کب:

- Information source document میں موجود □ی نہ□یں
- Required context external □□ (میں □□ جو فرا□م نہ□یں کیا گیا document کسی اور data)

Pydantic بطور validation tool: Pydantic Python library جو □□ schema-based data validation □لیے □کے استعمال □وتی □□□ امتحان کے □لیے ا□م نکات یہ □یں:

- **Structural validation:** types، requiredness، enum constraints JSON بعد □کے code □یں □جاتے □یں چیک کیے میں
- **Semantic validation:** custom validators business logic enforce □یں □کرتے (sum of items equals total؛ start_date < end_date)
- **Validate–retry loops:** Pydantic validation failure پر error message بنا کر Claude کو error context □کے ساتھ prompt کریں دوبار□
- **JSON Schema generation:** Pydantic models `tool_use` □لیے □کے JSON Schema generate □کر سکتے □یں، جو ایک single source of truth □□ فرا□م کرتا

6.6 Self-correction

Internal contradictions detect کا □کرنے pattern:

```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

آپ `conflict_detected` دونوں نکالتا □□ — اگر وہ □ مختلف □وں تو Model stated value اور computed value کو discrepancy handle □□□ دیتا □کرنے

باب 7: Message Batches API

> دستاویزات: Message Batches

7.1 Overview

Message Batches API آپ کو asynchronous processing □لیے □کے requests □کے batches submit □□ دیتا □کرنے:

Attribute	Value
Savings	synchronous calls **50** میں مقابلہ میں 50%
Processing window	**24** گھنٹے سے زیادہ **hours** (latency SLA guarantee نہیں)
Multi-turn tool calling	**Supported** نہیں (ایک request = ایک response)
Correlation	request اور response کو جوڑنے کے لیے `custom_id` field

7.2 Batch API بمقابلہ Synchronous API کب استعمال کریں

Task	API	Why
Pre-merge PR check	**Synchronous**	Developer 24 hours انتظار کر رہا ہے؛ قابلِ قبول نہیں
Overnight tech-debt report	**Batch**	Result 50% savings صبح تک چاہیے؛
Weekly security audit	**Batch**	50% savings فوری نہیں؛
Interactive code review	**Synchronous**	فوری response درکار ہے
10,000 documents process کرنا	**Batch**	Bulk processing؛ savings اہم ہیں

7.3 `custom_id` کا استعمال

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract data from: ..."}]
  }
}
```

`custom_id` آپ کو یہ کرنے دیتا ہے:

- Result کو original document سے link کرنا
- Failure کی صورت میں صرف failed documents دوبارہ submit کرنا
- Successful documents کو دوبارہ process سے بچنا کرنے

7.4 Batch میں Failures کو Handle کرنا

1. 100 documents کا batch submit کریں
2. 95 succeed 5 fail کریں؛ (context limit exceeded) ہوں
3. Failures کو `custom_id` سے identify کریں
4. Strategy بدلیں (مثلاً long documents کو chunks میں تقسیم کریں)

5. 5 صرف اُن failed documents □کو دوبار□ submit کریں

7.5 SLA Planning

30 اگر آپ کو hours میں result اور □چا□ی□ Batch API 24 کو hours □یں □تک لگ سکت□:

- Submission window: 30 - 24 = **6 hours**
- Batches کو deadline 24 کم از کم □س hours □ل□ پ submit □وگا کرنا
- Frequent submissions 4 □ل□ کی-hour windows میں تقسیم کریں

---

8 باب: Task Decomposition Strategies

8.1 Fixed Pipelines (Prompt Chaining)

□ر step □س □ل□ پ defined □□ □وتا:

```
Document -> Metadata extraction -> Data extraction -> Validation -> Enrichment -> Final
output
```

کب استعمال کریں:

- Task structure predictable □و (reviews □ی □ایک □میش□ template follow کریں)
- تمام steps □وں □معلوم □س □ل□ پ
- آپ کو stability اور reproducibility □چا□ی□

8.2 Dynamic Adaptive Decomposition

Subtasks intermediate results پر کی بنیاد generate □یں □وت□:

```
1. "Legacy codebase کے لیے tests add کریں"
2. -> پہلے structure map کریں (Glob, Grep)
3. -> 3 :ملا modules بغیر tests 2 ،کے partial coverage کے ساتھ
4. -> Prioritize: payments module سے شروع کریں (high risk)
5. -> کام کے دوران: external API پر dependency دریافت ہوئی
6. -> Adapt: tests پہلے سے لکھنے external API کے لیے mock add کریں
```

کب استعمال کریں:

- Open-ended investigative tasks
- جب مکمل scope □و □ نہ شروع میں معلوم
- □ر جب step □پچھل step □کے results □و پر منحصر

8.3 Multi-pass Code Review

10+ files والی pull requests □لیے □کے:

```
Pass 1 (per-file): auth.ts analyze کریں -> local issues کی list
Pass 1 (per-file): database.ts analyze کریں -> local issues کی list
Pass 1 (per-file): routes.ts analyze کریں -> local issues کی list
...
Pass 2 (integration): files کے درمیان relationships analyze کریں
  -> Cross-file issues: inconsistent types, circular dependencies
```

14 files □ی پر ایک pass □□ کیوں خراب:

- Attention dilution: کچھ files کی deep analysis، کچھ کی shallow
- Inconsistent comments: ایک file میں pattern flag، دوسری میں approve
- Missed bugs: cognitive overload واضح □سے □وجہ کی errors □میں چھوٹ جاتے

---

9 باب: Escalation اور Human-in-the-Loop

9.1 کب Human کی طرف Escalate کریں

Escalation triggers (واضح rules):

Situation	Action
Customer □□کے واضح طور پر “get me a manager”	فوراً escalate کریں؛ حل کرنے کی کوشش نہ کریں
Policy request کو cover □و نہ کرتی	Escalate کریں (مثلاً competitor price matching جب policy خاموش □ہو)
Agent □کر سکے پیش رفت نہ کر	مناسب تعداد میں attempts کے بعد escalate کریں
Threshold □سے اوپر financial operation	Escalate کریں (ترجیحاً hook □کے ذریعے enforce، prompt □کے ذریعے نہیں)
Customer وقت □تلاش کرتے multiple matches	مزید identifiers مانگیں؛ اندازے نہ لگائیں

جو reliable trigger □□ نہیں:

Unreliable method	Why it fails
Sentiment analysis	Customer کا mood case complexity □سے correlate نہیں کرتا
Model self-rated confidence (1–10)	Model confidently □□غلط □ہو سکتا؛ calibration □□ کمزور
Automatic classifier	Overengineering□□ ممکن ؛ training data □ہو نہ موجود

9.2 Escalation Patterns

Immediate escalation:

```
Customer: "I want to speak to a manager"
Agent: [فوراً escalate_to_human call □□ کرتا]
NOT: "I can help with your issue, let me..."
```

Resolve بعد □ کی کوشش کرنا escalation:

```
Customer: "My refrigerator broke two days after purchase"
Agent: [order □□ چیک کرتا، warranty replacement offer □□ کرتا]
اگر customer □ مطمئن نہ □ -> escalate
```

Nuanced escalation (acknowledge → resolve → reiteration پر escalate):

```
Customer: "This is outrageous, I'm very unhappy with the quality!"
Agent: [frustration acknowledge] "میں آپ کی frustration □ سمجھتا □ں."
       [resolution offer] "میں replacement یا refund □ پیش کر سکتا □ں."
Customer: "No, I want to talk to someone!"
Agent: [customer □ دوبارہ insist □ کرے -> immediate escalation]
```

□□ پ: بنیادی اصول emotion acknowledge پھر، کریں concrete solution اور صرف اسی وقت، دیں escalate کریں جب customer human □□ کی خواہش د□رائ□ dissatisfaction پر اظ□ار □□ پ□ کے escalate کریں □ نہ (□ یہ manager □□ برابر نہیں □ کہ □ مانگنا□)

Policy gap □ لیے □ کے escalation:

```
Customer: "Competitor X has this item 30% cheaper—give me a discount"
Policy: صرف اپنی site پر price adjustments cover □□ کرتی
Agent: [escalate — policy competitor price matching کو cover نہیں کرتی]
```

9.3 Structured Handoff Protocols

Escalation پر agent کو structured summary human □□ کو دینی چاہیے:

```
{
  "customer_id": "CUST-12345",
  "customer_name": "Ivan Petrov",
  "issue_summary": "Refund request for a damaged item",
  "order_id": "ORD-67890",
  "root_cause": "Item arrived damaged; photos attached",
  "actions_taken": [
    "Verified customer via get_customer",
    "Confirmed order via lookup_order",
    "Offered a standard replacement — customer insists on a refund"
  ],
  "refund_amount": "$89.99",
  "recommended_action": "Approve a full refund",
  "escalation_reason": "Customer requested to speak with a manager"
}
```

Human operator کو full conversation transcript □ یہ صرف □ وہ — □وتی □یں نہ□یں تک رسائی summary □□□ دیکھتا اور مکمل □ یہ □ لیے اس self-contained □□یہ□ چاہ□ونی

9.4 Confidence Calibration اور Human Oversight

Data extraction systems □ لیے □کہ:

1. **Field-level confidence scores:** model □ر extracted field □ لیے □کہ confidence score □□ نکالتا
2. **Calibration:** labeled validation sets □کہ استعمال کر thresholds tune کریں
3. **Routing:**
   - High confidence + stable accuracy -> automated processing
   - Low confidence یا ambiguous sources -> human review

Stratified random sampling:

- High-confidence extractions □سے باقاعدگی بھی □ لیے □کہ sample audit کریں
- Aggregate 97% accuracy کسی مخصوص document type 40 □ لیے □کہ% errors □□ چھپا سکتی
- Accuracy کو صرف overall □یں نہ، document type اور field □سے حساب □کہ analyze کریں

---

10 باب: Multi-agent Systems میں Error Handling

10.1 Error Categories

Category	Examples	Retryable	Agent action
Transient	Timeout، 503، network failure	Yes	Exponential backoff ساتھ □کہ retry
Validation	Invalid input format، missing required field	No (input fix کریں)	Request modify کریں اور retry کریں

Business	Policy violation، threshold exceeded	No	User کو سمجھائیں؛ alternative پیش کریں
Permission	Access denied	No	Escalate کریں

10.2 Error-handling Anti-patterns

Anti-pattern	Problem	Correct approach
Generic status “search unavailable”	Coordinator decide □□یں کر سکتا کہ recover □□ کرنا □کیسے	Error type، query، partial results، alternatives واپس کریں
Silent suppression (empty result = success)	Coordinator □□ سمجھتا match □□یں ملا، حالانکہ failure تھا □وا	“no results” اور “search failure” میں فرق کریں
ایک failure پر پورا workflow abort کرنا	تمام partial results □□یں ضائع □و جاتے	Partial results □ساتھ جاری رکھیں؛ gaps annotate کریں
Subagent میں infinite retries	Latency اور resources کا ضیاع	Local recovery (1–2 retries)، پھر coordinator تک propagate

10.3 Structured Subagent Error

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    {"title": "AI Music Generation Report", "url": "...", "relevance": 0.8}
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

□ی coordinator □□ ضروری معلومات دیتا □لیے کہ □کو فیصلہ کرنے:

- Modified query □ساتھ retry کریں؟
- Partial results استعمال کریں؟
- کسی اور subagent کو delegate کریں؟
- اس section بغیر جاری رکھیں اور □کو gap annotate کریں؟

10.4 Final Synthesis میں Coverage Annotations

```
## Report: AI Impact on Creative Industries

### Visual Art (FULL COVERAGE)
[research results]

### Music (PARTIAL COVERAGE — search agent timeout)
[partial results]
⚠️ Note: اس section کی coverage search agent timeout □□ محدود □ سے وجہ کی.

### Literature (FULL COVERAGE)
[research results]
```

---

11 باب: Production Systems میں Context Management

11.1 Facts کو Separate Block میں Extract کریں

Conversation history □بجائ□ کہ □ پر انحصار کرنا (جو summarization دوران □کہ degrade □□ جاتی □ہو)، key facts کو structured block میں extract کریں:

```
=== CASE FACTS (جب بھی نیا fact آئے update کیا جائے) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Order Date: 2025-01-15
Order Amount: $89.99
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
```

کیوں نہ □ہو□ summarized کتنی □ہی history میں شامل کریں، چا□□ prompt □ہر block □یا

11.2 Tool Results کو Trimming کرنا

اگر `lookup_order` 40+ fields مگر □□ واپس کرتا current task □میں صرف 5 درکار □لیے □کہ:

```python
# PostToolUse hook: صرف relevant fields رکھیں
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

□س اس context اور □□ بچتا noise □□□ کم □وتی

11.3 Position-aware Input

Lost-in-the-middle effect □وئ□ □تھ□ میں رکھ□ن ذ□ کو critical information رکھیں:

```
[KEY FINDINGS — اوپر]
3 critical vulnerabilities ملیں...

[DETAILED RESULTS — درمیان]
=== File auth.ts ===
...
=== File database.ts ===
...

[ACTION ITEMS — آخر میں]
Priority: merge □ پ□ل□ □س auth.ts vulnerabilities fix کریں.
```

11.4 Scratchpad Files

Long investigations میں agent key findings scratchpad file □□ میں لکھ سکتا:

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() تین places □س call □□ □وتی: OrderController، AdminPanel، CronJob
- External PaymentGateway API limit: 100 req/min
- Migration #47 □ز refund_reason (NOT NULL) add 01-12-2024 — کیا
```

جب context degrade □جائ□ (نئی یا session میں)، agent discovery □بجائ□ □ک□ چلان□ دوبار□ scratchpad consult □□□ کر سکتا

11.5 Context □ لیے □ کے بچانے Subagents کو Delegate کرنا

```
Main agent: "payments module کی dependencies investigate کریں"
  -> Subagent (Explore): 15 files read □□ کرتا، imports trace □□ کرتا
  -> Returns: "Payments depends on AuthService, OrderModel, and the external
PaymentGateway API"

Main agent: context 15 میں files ایک بجائے کہ line □□ رکھتا
```

Separate context layer: Multi-agent systems ہر□ میں subagent محدود context budget — □□ میں کام کرتا □ کہ separate context layer پر طور ایک Coordinator ملتی □□□ information کے لیے □ ضروری task اسے □ صرف اپنے کرتا، global state store اور □□ کرتا context allocate کرتا subagent outputs aggregate □□ کرتا: □□ وہ □ اس سے □□□ "context leakage" ایک جو information کو ایسی agent window نہ□یں □وتا، جہاں □□ بھر دیتا سے □ دوسروں کے لیے غیر متعلق □وں

Subagents □ لیے □ کے constrained context budgets:

- Minimal context بھیجیں: specific task + ضروری data
- Subagent کو raw data dumps □ بجائے □ کے structured results واپس کرنے کی □ہدایت دیں
- `allowedTools` □ سے subagent □ کے toolset کو limit کم — کریں tools کم مطلب کا distractions اور کم context cost

11.6 Structured State Persistence (crash recovery □ لیے □ کے)

ہر□ agent اپنی state ایک معلوم location پر export □□ کرتا:

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

Coordinator resume پر ایک manifest load □□ کرتا:

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

12 باب: Provenance کو محفوظ رکھنا

12.1 Attribution Loss Problem

جب متعدد sources □سے results summarize □کیے جاتے □ہیں، تو “claim → source” link □□ گم □ہو سکتا:

```
Bad: "The AI music market is estimated at $3.2B." (No source, no year.)

Good:
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

12.2 Conflicting Data کو Handle کرنا

جب دو sources مختلف values دیں:

```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Methodology اور time period فرق کا"
}
```

کسی ایک value کو دونوں کو ساتھ محفوظ رکھیں اور attribution کو اندھا دھند منتخب نہ کریں□ coordinator کو فیصلہ کرنے دیں□

12.3 Correct Interpretation کے لیے Dates شامل کریں

Dates بغیر temporal differences کو contradiction □□ سمجھ لیا جا سکتا:

```
Bad: "Source A says 10%, source B says 15%. Contradiction."
Good: "Source A (2023) says 10%, source B (2024) says 15%. Likely 5+ ایک سال میں %
growth."
```

12.4 Content Type مطابق □کو Render کریں

□ہر چیز کو ایک □ی format میں □نہ ٹھونسیں:

- Financial data -> tables
- News اور analysis -> prose
- Technical findings -> structured lists
- Time series -> chronological ordering

---

13 باب: Claude Code □کے Built-in Tools

13.1 Tool Selection Reference

Task	Tool	Example
نام/pattern □سے files تلاش کرنا	**Glob**	`**/*.test.tsx`, `src/components/**/*.ts`
File contents اندر □کو search کرنا	**Grep**	Function name، error message، import
File مکمل پڑھنا	**Read**	Analysis □لیے □کو file load کرنا
نیا file بنانا	**Write**	Scratch □سے file create کرنا
□موجود file میں precise edit	**Edit**	Unique text match □ذریعے □کو specific snippet replace کرنا
Shell command چلانا	**Bash**	git، npm، tests، build

13.2 Incremental Investigation Strategy

□نہ پڑھیں□ سمجھ بوجھ کو رفتہ رفتہ□ بنائیں files ایک ساتھ تمام:

```
1. Grep: entry points تلاش کریں (function definition، export)
2. Read: ملنہ والی files پڑھیں
3. Grep: usages تلاش کریں (import، calls)
4. Read: consumer files پڑھیں
5. جب تک مکمل picture نہ مل جائہ repeat کریں
```

13.3 Fallback: Edit □بجائ □کو Read + Write

جب Edit non-unique text match □سے □وجہ کی fail □جائو□:

1. Read — مکمل file content لوڈ کریں
2. Content کو programmatically modify کریں
3. Write — updated version لکھ دیں

□حصہ II: امتحانی ڈومین نوٹس

ڈومین 1: Agent Architecture اور Orchestration (27%)

1.1 Autonomous Task Execution □لیے □کے Agentic Loops Design کرنا

Key knowledge:

- Agent loop lifecycle: Claude request بھیجیں، `stop_reason` (`"tool_use"` بمقابلہ□ `"end_turn"`) چیک کریں، tools execute کریں، اگلی iteration □لیے □کے results واپس کریں
- Tool results conversation history میں append □کریں تاکہ □وتے□ model اگلا action □کر سکے□ طے□
- Model-driven decision making (Claude اگلا tool □□ چنتا) بمقابلہ□ hard-coded decision trees

Key skills:

- Flow control: جب `stop_reason = "tool_use"` □ہو تو loop جاری رکھیں اور `"end_turn"` پر رک جائیں
- Iterations درمیان □کے tool results کو context میں append کرنا
- Anti-patterns بچنا □سے: completion □لیے □کے assistant text parse کرنا، primary stopping mechanism طور □کے پر arbitrary iteration limits استعمال کرنا

1.2 Multi-agent Systems (Coordinator–Subagent) Orchestrate کرنا

Key knowledge:

- Hub-and-spoke architecture: coordinator تمام inter-agent communication، error handling، اور routing کا □□ مالک □وتا
- Subagents isolated context خود بخود □ہیں — وہ □نہیں کرتے□ میں کام کرتے□ coordinator کی history inherit
- Coordinator responsibilities: task decomposition، delegation، result aggregation، dynamic selection of subagents
- Coordinator کی overly narrow decomposition □کا خطرہ

Key skills:

- Research coverage کو subagents □کے درمیان تقسیم کریں تاکہ□ duplication □کو کم

- Iterative refinement loops implement کریں (coordinator synthesis evaluate اور کر tasks re-route کر)
- Observability تمام لیں کہ communication coordinator ذریعہ کہ route کریں

1.3 Subagent Calls، Context Passing، اور Spawning Configure کرنا

Key knowledge:

- `Task` tool subagents spawn کرتا ہے؛ coordinator کہ `allowedTools` میں `"Task"` شامل ہونا چاہیے
- Subagent context prompt میں explicitly شامل ہونا چاہیے؛ subagents parent context inherit نہیں کرتے
- `AgentDefinition` configuration: descriptions، system prompts، tool constraints
- Alternatives explore کرنے کے لیے `fork_session` ذریعہ کہ session management

Key skills:

- Previous agents کہ full outputs subagent prompt میں شامل کریں
- Context pass وقت کرتے data اور metadata کو separate کرنے کے لیے structured formats استعمال کریں
- ایک coordinator turn میں متعدد `Task` calls ذریعہ کہ parallel subagents spawn کریں
- Coordinator prompts goals اور quality criteria طور پر لکھیں کہ، step-by-step instructions کہ طور پر نہیں

1.4 Enforcement اور Handoff Patterns کہ ساتھ Multi-step Workflows Implement کرنا

Key knowledge:

- Workflow ordering لیے کہ **programmatic enforcement** (hooks، preconditions) اور **prompt guidance** کہ درمیان فرق
- جب deterministic guarantees درکار ہوں (مثلاً مالی operations پہلے سے identity verification)، prompts کافی نہیں ہوتے
- Escalation دوران کہ structured handoff protocols (customer ID، reason، recommended action)

Key skills:

- ایسے programmatic preconditions جو downstream calls کو block کریں جب تک پہلے steps مکمل نہ ہوں (مثلاً `get_customer` کہ verified ID تک واپس کرنے `process_refund` block کریں)
- Multi-aspect customer requests کو الگ items میں تقسیم کریں
- Human کو escalate وقت کرتے structured summaries produce کریں

1.5 Tool Calls Intercept اور کرنے Data Normalize لیے کہ کرنے Agent SDK Hooks

Key knowledge:

- Hook patterns (مثلاً `PostToolUse`) tool results کو model کہ consume پہلے سے کرنے intercept میں کرتے

- Hooks outgoing calls intercept کر کے compliance rules enforce ہیں کرتے (مثلاً threshold اوپر سے refunds block کرنا)
- Hooks **deterministic guarantees** دیتے ہیں، جبکہ prompt instructions **probabilistic compliance** دیتی ہیں

Key skills:

- `PostToolUse` hooks سے data formats normalize کریں (Unix timestamps، ISO 8601، numeric status codes)
- Policy-violating actions block کرنے کے لیے interception hooks استعمال کریں اور escalation کی طرف redirect کریں
- جب business rules کو guaranteed compliance ہو تو prompts بجائے hooks منتخب کریں چاہیے

1.6 Complex Workflows کے لیے Task Decomposition Strategies

Key knowledge:

- **Fixed pipelines** (prompt chaining) بمقابلہ intermediate results کی بنیاد پر **dynamic adaptive decomposition**
- Prompt chaining: sequential steps (ہر file کو الگ analyze کریں، پھر integration pass)
- Adaptive investigation plans جو discovered information کی بنیاد پر subtasks generate کرتے ہیں

Key skills:

- Predictable multi-aspect reviews کے لیے prompt chaining؛ open-ended investigations کے لیے dynamic decomposition استعمال کریں
- Large code reviews کو per-file analysis + separate cross-file integration pass میں تقسیم کریں
- Open-ended tasks کو degrade کریں: پہلے structure map کریں، پھر prioritized plan بنائیں

1.7 Session State، Resuming، اور Forking

Key knowledge:

- Named sessions continue کرنے کے لیے `--resume <session-name>`
- Shared context سے independent investigation branches بنانے کے لیے `fork_session`
- Resume وقت کرتے agent کو file changes کی اہمیت بتانے
- Structured summary ساتھ نئی session، stale results ساتھ resuming کے زیادہ سے reliable ہو سکتی ہے

Key skills:

- Named investigation sessions continue کرنے کے لیے `--resume` استعمال کریں
- Approaches parallel compare کرنے کے لیے `fork_session` استعمال کریں
- Resuming (context ابھی current ہے) اور نئی session شروع کرنے (results stale ہیں) کے درمیان انتخاب کریں

5.10 `fork_session` اور Session Management

`--resume <session-name>` named session کو resume □□ کرتا:

```
claude --resume investigation-auth-bug
```

- saved context پچھلی ساتھ □ کے conversation □□ جاری رکھتا
- Long investigations جو متعدد sessions مفید □ لیے □وں ان کے پر پھیلی □وں
- Risk: اگر پچھلی session بعد □ کے files تو □وں بدل چکی tool results stale میں □ سکتے □و

`fork_session` shared context □ سے independent branch □□ بناتا:

```
Codebase investigation
         |
    fork_session
    /           \
Approach A:      Approach B:
Redux            Context API
```

- دونوں forks branch point تک context inherit میں □ کرتے
- طور پر آزادانہ □ بعد وہ □ کے اس diverge میں □ کرتے
- Approaches compare یا □ کرنے strategies test مفید □ لیے کے کرنے

Resume نئی □ بجائے □ کے session کب شروع کریں:

- Tool results stale □وں (files □وں بدل چکی)
- اور □و کافی وقت گزر چکا context degrade □و گیا □و
- پرانے tool data ساتھ □ کے resume: □□ یہ □ جو پایا اس کا مختصر خلاصہ □ "□م نے □ کے □تر □□ بہ یہ بجائے □ کے کرنے restart □ کیا جائے □ سے "...

---

2 ڈومین: Tool Design اور MCP Integration (18%)

2.1 Clear Descriptions ساتھ □ کے Tool Interfaces Design کرنا

Key knowledge:

- Tool descriptions **بنیادی** وہ **mechanism** □ جن سے □ میں LLM tools select کرتا □□؛ minimal descriptions unreliable selection میں □ دیتی
- Input formats، example queries، edge cases، اور applicability boundaries □میت کی □ شامل کرنے
- Ambiguous یا overlapping descriptions misrouting میں □ کا سبب بنتی
- System prompt wording tools غیر ارادی ساتھ □ کے associations □□ بنا سکتی

Key skills:

- ایسی descriptions □ر لکھیں جو tool کو similar alternatives واضح طور پر □ سے distinguish کریں

- Functional overlap □لیے □کے ختم کرنے tools rename کریں (مثلاً `analyze_content` -> `extract_web_results`)
- General-purpose tools کو specialized tools □کے جن میں تقسیم کریں input/output contracts واضح □وں

2.2 MCP Tools □لیے □کے Structured Error Responses Implement کرنا

Key knowledge:

- MCP tool responses میں `isError` flag
- **Transient errors** (timeouts)، **validation errors** (bad input)، **business errors** (policy violations)، اور **access/permission errors** درمیان فرق □کے
- Generic errors ("Operation failed") صحیح recovery decisions □میں روک دیتے
- Retryable اور non-retryable errors میں فرق

Key skills:

- `errorCategory` (transient/validation/permission)، `isRetryable`، اور human-readable message جیسی structured metadata واپس کریں
- Business-rule violations واضح □لیے □کے user-facing explanations ساتھ □کے `retryable: false` استعمال کریں
- Transient failures □لیے □کے subagents میں local recovery □وہ صرف کریں؛ errors propagate خود □وہ جو کریں حل نہ □کر سکیں
- Access failures (retry decision) اور valid empty results (no matches) میں فرق کریں

2.3 Agents میں Tools Allocation اور `tool_choice` Configure کرنا

Key knowledge:

- □زیادہ tools (5–4 □کے بجائے 18 مثلاً) tool selection reliability □□ کم کرتی
- Specialization □سے باہر tools □والے رکھنے agents □میں غلط استعمال کرتے □ہیں
- Scoped tool access: صرف role-relevant tools + محدود cross-role utilities
- `tool_choice`: `"auto"`, `"any"`, اور forced tool selection (`{"type": "tool", "name": "..."}`)

Key skills:

- □ہر subagent کا toolset صرف relevant tools تک محدود رکھیں
- General tools کو constrained alternatives بدلیں □سے (مثلاً `fetch_url` -> `load_document`)
- `tool_choice: "any"` □استعمال کریں تاکہ text answer □بجائے □کے tool call □ہو یقینی
- Specific tool force □کریں تاکہ execution order assured □ہو

2.4 MCP Servers کو Claude Code اور Agent Workflows میں Integrate کرنا

Key knowledge:

- MCP server scope: ٹیموں کے لیے project (`.mcp.json`) بمقابلہ experiments کے لیے user (`~/.claude.json`)
- Secrets management کے لیے `.mcp.json` میں environment variable substitution (مثلاً `${GITHUB_TOKEN}`)
- تمام connected MCP servers کے tools connection پر discover ہو جاتے ہیں اور ایک ساتھ available ہوتے ہیں
- MCP resources بطور "content catalogs" (task summaries، database schemas) exploratory tool calls کم کرتے ہیں

Key skills:

- Shared MCP servers کو project `.mcp.json` میں env-var-based tokens کے ساتھ configure کریں
- Personal/experimental servers کو `~/.claude.json` میں رکھیں
- Standard integrations کے لیے community MCP servers کو custom servers پر ترجیح دیں

2.5 Built-in Tools (Read, Write, Edit, Bash, Grep, Glob) منتخب اور Apply کرنا

Key knowledge:

- **Grep**: file contents اندر کے search (function names، error messages، imports)
- **Glob**: name/extension patterns سے files تلاش کرنا
- **Read/Write**: full-file operations؛ **Edit**: unique text matches کے ذریعے precise changes
- اگر Edit non-unique matches سے وجہ کی fail ہو تو Read + Write پر واپس جائیں

Key skills:

- Content search کے لیے Grep اور pattern-based discovery کے لیے Glob استعمال کریں
- سمجھ بوجھ رفتہ رفتہ بنائیں: entry points کے لیے Grep، پھر flows trace کے لیے کرنے Read
- Wrapper modules کے ذریعے function usage trace کریں

ڈومین 3: Claude Code Configuration اور Workflows (20%)

3.1 Hierarchy، Scope، اور Modular Organization کے ساتھ CLAUDE.md Configure کرنا

Key knowledge:

- CLAUDE.md hierarchy: user (`~/.claude/CLAUDE.md`)، project (`.claude/CLAUDE.md` یا root `CLAUDE.md`)، اور directory-level (subdirectories میں CLAUDE.md)
- User-level settings صرف ایک user اور ہوتے ہیں پر لاگو ہوتے ہیں VCS کے ذریعے share نہیں
- External files refer کرنے کے لیے `@path` syntax (مثلاً `@./standards/coding-style.md`) تاکہ CLAUDE.md modular بنے
- Monolithic CLAUDE.md کے بجائے topic-focused rule files کے لیے `.claude/rules/` directory

Key skills:

- Hierarchy issues diagnose کریں (نئے team member کو instructions وہ نہیں ملتیں کیونکہ اس لیے user-level میں تھیں، project-level میں نہیں)
- ہر package کے CLAUDE.md میں standards selectively include کرنے کے لیے `@path` (مثلاً `@./standards/testing.md`) استعمال کریں
- بڑے CLAUDE.md کو متعدد `.claude/rules/` files (testing.md، api-conventions.md، deployment.md) میں تقسیم کریں

3.2 Custom Slash Commands اور Skills بنانا اور Configure کرنا

Key knowledge:

- `.claude/commands/` میں **Project commands** (VCS کے ذریعے shared) بمقابلہ `~/.claude/commands/` میں **user commands**
- `.claude/skills/` میں SKILL.md frontmatter: `context: fork`، `allowed-tools`، `argument-hint`
- `context: fork` skill کو isolated subagent context تاکہ وہ چلاتا ہے میں main session pollute نہ ہو
- Personal skill variants `~/.claude/skills/` میں مختلف ناموں سے رکھ سکتے ہیں

Key skills:

- Project slash commands `.claude/commands/` میں رکھیں تاکہ پوری team انہیں حاصل کرے
- Verbose output والے skills isolate کرنے کے لیے `context: fork` استعمال کریں
- Skill کے استعمال کردہ tools کو محدود کرنے کے لیے `allowed-tools` استعمال کریں
- Required parameters مانگنے کے لیے `argument-hint` استعمال کریں

3.3 Conditional Convention Loading کے لیے Path-specific Rules استعمال کرنا

Key knowledge:

- `.claude/rules/` files میں YAML frontmatter `paths` کے تاکہ وہ شامل ہو سکتا ہے glob patterns کے مطابق rules activate ہوں
- Path-scoped rules صرف matching files edit وقت کرتے load ہیں ہوتے ہیں، context اور tokens بچتے ہیں
- Glob-based path rules directory-level CLAUDE.md جب ہیں سے بہتر ہو سکتے ہیں کئی conventions directories میں apply ہوں (مثلاً tests)

Key skills:

- `paths: ["terraform/**/*"]` کے ساتھ `.claude/rules/` files صرف بنائیں تاکہ matching files پر load ہوں
- Glob patterns (`**/*.test.tsx`) استعمال کریں تاکہ location قطع نظر سے file type کے لحاظ سے conventions apply ہوں
- جب conventions codebase بھر میں پھیلی ہوں تو directory-level CLAUDE.md بجائے کے path-specific rules کو ترجیح دیں

3.4 Planning Mode بمقابلہ Direct Execution کب استعمال کریں

Key knowledge:

- **Planning mode**: complex tasks، بڑے changes، multiple viable approaches، اور architectural decisions کے لیے
- **Direct execution**: سادہ، اچھی طرح سمجھے گئے changes (مثلاً single validation شامل کرنا)
- Planning mode changes پہلے سے codebase کی safe exploration کے ممکن بناتا ہے
- Explore subagent verbose discovery output isolate کرتا ہے

Key skills:

- Architectural consequences والے tasks (microservices، 45+ files والی migrations) کے لیے planning mode استعمال کریں
- واضح stack trace اور single file والے fixes کے لیے direct execution استعمال کریں
- Multi-phase tasks میں context-window exhaustion کے لیے روکنے Explore subagent استعمال کریں
- Approaches کو combine کریں: discovery کے لیے plan، implementation کے لیے execute

3.5 Progressive Improvement کے لیے Iterative Refinement

Key knowledge:

- Concrete input/output examples expectations communicate کرنے کا سب سے مؤثر طریقہ ہیں
- **Test-driven iteration**: پہلے tests لکھیں، پھر failures کی بنیاد پر iterate کریں
- "Interview" pattern: Claude clarifying questions پوچھتا ہے تاکہ non-obvious design considerations سامنے آئیں

- تمام issues ایک message میں کب دیں (interdependent) اور sequentially کب (independent)

Key skills:

- Transformation requirements 3–2 کے لیے واضح کرنے کے concrete input/output examples فراہم کریں
- Implementation پہلے سے expected behavior، edge cases، اور performance requirements کے ساتھ test sets بنائیں
- Design aspects (cache invalidation، failure modes) کے لیے سامنے لانے کے interview pattern استعمال کریں
- Edge cases کے لیے sample inputs اور expected outputs کے ساتھ concrete test cases دیں

3.6 Claude Code کو CI/CD Pipelines میں Integrate کرنا

Key knowledge:

- Automated pipelines میں non-interactive mode کے لیے `-p` (یا `--print`) flag
- CI میں structured output کے لیے `--output-format json` اور `--json-schema`
- CLAUDE.md project context (testing standards، review criteria) فراہم کرتا ہے
- **Session context isolation**: code generate کرنے والی same session اسے review ہوتی ہے کرنے میں کم مؤثر ہے

Key skills:

- Interactive input پر hang ہونے سے بچنے کے لیے CI میں Claude Code کو `-p` کے ساتھ چلائیں
- Structured results (مثلاً inline PR comments) کے لیے `--output-format json` + `--json-schema` استعمال کریں
- نئے commits کے بعد دوبارہ run کرتے وقت پچھلے review results شامل کریں (صرف new/unfixed issues report کریں)
- Test generation quality کے لیے بہتر کرنے کے CLAUDE.md میں testing standards اور available fixtures document کریں
- نئے tests generate کرتے وقت existing test files کو context میں شامل کریں تاکہ style، naming اور duplication consistent رہے

---

4 ڈومین: Prompt Engineering اور Structured Output (20%)

4.1 Accuracy کے لیے بہتر کرنے کے Explicit Criteria کے ساتھ Prompts Design کرنا

Key knowledge:

- Explicit criteria vague instructions میں سے زیادہ مؤثر ہوتے ہیں (مثلاً “flag comments only when they contradict code” بمقابلہ “check comment accuracy”)
- “be more conservative” جیسی generic guidance concrete categorical criteria سے کم مؤثر ہے

- False positives developer trust کچھ: درست high false-positive rates میں categories پر اثر انداز ہوتے ہیں کم کر دیتی ہیں trust پر بھی categories

Key skills:

- Review criteria define کریں: کیا report کرنا (bugs، security) اور کیا ignore کرنا (minor style) ہے
- High false-positive rates والی categories پر عارضی طور پر disable کریں
- ہر level کے لیے code examples کے ساتھ explicit severity criteria define کریں

4.2 Output Consistency بہتر کرنے کے لیے Few-shot Prompting استعمال کرنا

Key knowledge:

- Few-shot examples consistently formatted، actionable output مؤثر پیدا کرنے کا سب سے method ہیں
- Few-shot مبہم cases (tool selection، test coverage gaps) کو دکھا سکتا ہے
- Few-shot model کو نئے patterns پر generalize، صرف دہرانے میں نہیں defaults کرنے میں مدد دیتا ہے
- Few-shot extraction tasks میں hallucinations کم کر سکتا ہے

Key skills:

- Ambiguous scenarios کے لیے rationale 2–4 کے ساتھ targeted examples دیں
- Output format (location، issue، severity، suggested fix) والے دکھانے examples شامل کریں
- ایسے examples جو دیں acceptable code patterns اور real issues میں فرق دکھائیں
- مختلف structures والے documents سے درست extraction کے examples دیں

4.3 `tool_use` اور JSON Schemas کے ساتھ Structured Output Enforce کرنا

Key knowledge:

- `tool_use` with JSON Schemas schema-conformant output اور JSON syntax errors ختم کی ضمانت دینے کا سب سے reliable طریقہ ہے
- `tool_choice: "auto"` میں model text واپس کر سکتا ہے؛ `"any"` لازماً tool call کرنی ہوتی ہے میں اسے منتخب کرتی ہے specific tool forced selection؛
- Strict JSON Schemas syntax errors ختم کرتی ہیں مگر semantic errors روکتیں نہیں (totals؛ ملتے نہیں fields میں غلط values)
- Schema design: required vs optional fields؛ enums کے ساتھ “other” + detail string extensibility کے لیے

Key skills:

- JSON Schemas کے ساتھ extraction tools define کریں اور `tool_use` results سے data parse کریں
- متعدد schemas ہوں تو structured output کے لیے یقینی بنانے کے لیے `tool_choice: "any"` استعمال کریں
- Specific tool call force کریں: `tool_choice: {"type": "tool", "name": "extract_metadata"}`
- Source میں information کی صورت میں نہ ہونے values fabricate کے لیے سے بچانے ہونے fields optional/nullable رکھیں

- Extensible categorization □لیے □کے `"unclear"` اور `"other"` □جیسے enum values + detail fields استعمال کریں

4.4 Extraction Quality □لیے □کے Validation، Retries، اور Feedback Loops Implement کرنا

Key knowledge:

- Retry-with-error-feedback: correction guide □لیے □کے □کرنے retry prompt میں concrete validation errors شامل کریں
- اگر information source □تو □نہ □ہی موجود retries میں □ہوتے □فائدے □بے
- Feedback loop design: finding کو trigger والا □کرنے pattern (`detected_pattern`) track کریں
- Semantic errors (totals reconcile □نہیں □ہوتے) □بمقابلے syntax errors (جو `tool_use` □سے address □ہوتے □ہیں)

Key skills:

- Original document، incorrect extraction، اور specific validation errors ساتھ □کے follow-up prompts دیں
- جب retry □کو □فائدے □بے (required info صرف external document □میں) تو اس کی شناخت کریں
- False positives analyze □لیے □کے □کرنے findings میں `detected_pattern` fields شامل کریں
- Discrepancies detect □لیے □کے □کرنے `calculated_total` اور `stated_total` دونوں extract □کے کر self-correction design کریں

4.5 Efficient Batch Processing Strategies Design کرنا

Key knowledge:

- Message Batches API: 50% savings، 24-hour تک processing window، latency SLA guarantee نہیں
- Batch processing non-blocking tasks (overnight reports، audits) اور □□ مناسب □لیے □کے blocking tasks (pre-merge checks) □لیے □نہیں
- Batch API ایک single request میں multi-turn tool calling support کرتا □نہیں
- `custom_id` fields batch اندر □کے request/response correlate □میں □کرتی

Key skills:

- Blocking checks □لیے □کے synchronous API اور overnight/weekly workloads □لیے □کے Batch API استعمال کریں
- SLA needs مطابق □کے batch submission cadence plan کریں (مثلاً 30-hour guarantee 24 □لیے □کے-hour processing 4 ساتھ □کے-hour windows)
- Failed documents (identified by `custom_id`) □صرف □انہی کو دوبارہ submit □کر کے failures handle کریں
- Large-scale processing □پہلے □سے sample ساتھ □کے prompts iterate کریں

4.6 Multi-instance اور Multi-pass Review Architectures Design کرنا

Key knowledge:

- Self-review limitations: model اپنی reasoning context کو فیصلوں اور اپنے ساتھ رکھتا ہے challenge کا کرنے امکان کم ہوتا ہے
- Independent review instances (generation context بغیر کے) subtle issues میں بہتر ہوتی ہیں تلاش کرنے میں
- Multi-pass review: per-file local analysis + cross-file integration pass attention dilution سے بچاتا ہے

Key skills:

- Changes review کے لیے کرنے کے generation context بغیر کے second independent Claude instance استعمال کریں
- Multi-file reviews کو per-file passes + integration passes میں تقسیم کریں تاکہ cross-file dataflow analysis ہو سکے
- Self-rated confidence کے ساتھ verification passes کے استعمال کر کے reviews کو calibrated سے طریقے route کریں

ڈومین 5: Context Management اور Reliability (15%)

5.1 Critical Information محفوظ رکھنے کے لیے Conversation Context Manage کرنا

Key knowledge:

- Progressive summarization خطرات کے: numeric values، percentages، اور dates vague summaries میں بدل جاتے ہیں
- Lost-in-the-middle effect: models long inputs کے شروع اور آخر کو زیادہ reliable سے طریقے process کرتے ہیں، لیکن درمیان کے findings miss کر سکتے ہیں
- Tool outputs relevance میں مقابلے کے context میں disproportionally جمع ہو سکتے ہیں (40+ fields 5 جب درکار ہوں)
- Subsequent API requests میں full conversation history کی اہمیت بھیجنے

Key skills:

- Transactional facts کو summarized history ایک باہر سے persistent “case facts” block میں نکالیں
- Verbose tool outputs کو relevant fields تک trim کریں
- Key findings کو aggregated data شروع میں کے explicit section headings کے ساتھ رکھیں
- Subagents سے structured outputs میں metadata (dates، sources) شامل کروائیں

5.2 Effective Escalation Patterns Design کرنا اور Ambiguity Resolve کرنا

Key knowledge:

- Suitable escalation triggers: human کی explicit request، policy gaps/exceptions، پیش رفت نہ ہو پانا
- Immediate escalation (explicit request) بمقابلہ attempt-to-resolve (agent scope اندر کہ)
- Sentiment analysis اور model confidence self-ratings case complexity کہ unreliable proxies ہیں
- Multiple customer matches لیے کہ heuristic guessing بجائے کہ additional identifiers مانگنے چاہئیں

Key skills:

- System prompt میں explicit escalation criteria few-shot examples دیں کہ ساتھ
- Human کی explicit request کو بغیر اضافی investigation فوراً کہ execute کریں
- جب policy کسی specific request لیے کہ ambiguous یا silent ہو تو escalate کریں
- Tool results میں multiple matches ہوں تو additional identifiers مانگیں

5.3 Multi-agent Systems میں Error Propagation Strategies Implement کرنا

Key knowledge:

- Structured error context (failure type، query، partial results، alternatives) coordinator کو smarter recovery دیتا ہے
- Access failures (timeouts retry decision چاہتے ہیں) اور valid empty results (no matches) میں فرق کریں
- Generic error statuses (“search unavailable”) coordinator سے valuable context چھپا دیتے ہیں
- Silent suppression یا ایک failure پر پورا workflow abort کرنا دونوں anti-patterns ہیں

Key skills:

- Structured error context واپس کریں: failure type، کیا attempt کیا گیا، partial results، possible alternatives
- Access failures کو valid empty results سے الگ کریں
- Transient failures کے لیے subagents میں local recovery صرف کریں؛ non-recoverable errors partial results ساتھ کے propagate کریں
- Synthesis میں coverage annotate کریں: کیا اچھی طرح supported ہے اور کہاں gaps ہیں باقی

5.4 Large Codebases Investigate کرتے وقت Context Efficiently Manage کرنا

Key knowledge:

- Long sessions میں context degradation: model unstable answers اور مخصوص classes کے دینے لگتا ہے بجائے “typical patterns” دیتا ہے کا حوالے
- Scratchpad files key findings کو context boundaries میں کے پار محفوظ رکھتے ہیں
- Subagents کو delegate سے کرنے verbose discovery output isolate ہے جاتی ہو

- Structured state persistence crash recovery □□ ممکن بناتی

Key skills:

- Specific questions □لیے □کے subagents spawn □کریں جبکہ high-level coordination main agent میں رکھیں
- Key findings محفوظ رکھنے□ اور بعد میں refer □لیے □کے کرنے□ scratchpad files استعمال کریں
- □اگلے phase □کے subagents spawn □پہلے □سے کرنے□ key findings summarize کریں
- Long investigations دوران □کے context usage □لیے □کے کم کرنے□ `/compact` استعمال کریں

5.5 Human Oversight اور Confidence Calibration □کے ساتھ Workflows Design کرنا

Key knowledge:

- Aggregate metrics (97 مثلاً% overall accuracy) specific document types یا fields □پر کمزوری چھپا سکتے□ ہیں
- Stratified random sampling high-confidence extractions میں error rates measure □□ کرتا
- Field-level confidence calibration labeled validation sets □کے ذریعے
- Automation □پہلے □سے document type اور field segment □کے حساب سے accuracy validate کریں

Key skills:

- New error patterns detect □لیے □کے کرنے□ stratified random sampling implement کریں
- Stable performance validate □لیے □کے کرنے□ document type اور field □کے حساب سے accuracy analyze کریں
- Field-level confidence scores output اور کریں labeled data □کے ذریعے review thresholds calibrate کریں
- Low-confidence یا ambiguous-source extractions human review کی طرف route کریں

5.6 Multi-source Synthesis میں Provenance Preserve کرنا اور Uncertainty Handle کرنا

Key knowledge:

- Summarization دوران □کے attribution اگر □□ کھو جاتی “claim → source” mappings محفوظ نہ رہیں
- Aggregation دوران □کے structured mappings برقرار رہنی چاہئیں
- Conflicting statistics کو arbitrarily ایک value □بجائے □کے منتخب کرنے□ attribution ساتھ □کے conflict annotate کر □کے handle کریں
- Temporal differences کو contradiction □لیے □کے بچنے□ سے سمجھنے□ publication/collection dates شامل کریں

Key skills:

- Subagents □سے “claim → source” mappings (URL، document name، quotes) output کروائیں
- Reports کو stable findings اور disputed ones میں الگ structure کریں
- Conflicting values کو annotations اور محفوظ رکھیں ساتھ □کے reconciliation □لیے □کے coordinator کو دیں
- Correct temporal interpretation □لیے □کے publication dates شامل کریں

- Content type مطابق □ کو render کریں: financial data tables میں، news prose میں، technical findings structured lists میں

Claude Certified Architect — Foundations Certification

مطالعہ□ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

Claude Certified Architect — Foundations سرٹیفیکیشن اس بات کی تصدیق کرتی □□ کہ□ ایک ماہ□ر Claude-based وقت درست حل بناتہ□ trade-off امتحان □□□ یہ□ فیصلہ□ کر سکتا Claude Code، Claude Agent SDK، اور Model Context Protocol (MCP) جن — یعنی وہ□ بنیادی ٹیکنالوجیز □□ کہ□ بنیادی علم کو جانچتا Claude سہ□ کہ□ ساتھ پروڈکشن ایپلیکیشنز تیار کی جاتی □یں□

بنانا، agentic systems امتحان کہ□ سوالات حقیقی دنیا کہ□ منظرناموں پر مبنی □وتہ□ □یں: کسٹمر سپورٹ کہ□ لیہ□ developer productivity، میں شامل کرنا CI/CD کو Claude Code، ڈیزائن کرنا multi-agent research pipelines، structured data نکالنا□ بنانا، اور غیر ساختہ□ دستاویزات سہ□ tools

---

□دف امیدوار

مثالی امیدوار ایک **solution architect** جو □□ Claude اور ڈیزائن ایپلیکیشنز پروڈکشن ساتھ کہ□ ship کرتا □□□ آپ کہ□ پاس کم از کم 6 ماہ□ کا عملی تجربہ□ □ونا چاہ□یہ□:

- **Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- **Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- **Model Context Protocol (MCP)** — backend integration □یہ□ کہ□ tools اور resources
- **Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- **Context windows** — لمبی دستاویزات، multi-agent context passing
- **CI/CD pipelines** — automated code review، test generation
- **Escalation and reliability** — error handling، human-in-the-loop

---

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (4 درست 1 □میں سہ□)

اسکورنگ	100–1000 scale، passing score **720**
Guessing penalty	نہیں (ہر سوال کا جواب دیں!)
Scenarios	6 4 میں سے (randomly selected)

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	**27%**
2. Tool design and MCP integration	**18%**
3. Claude Code configuration and workflows	**20%**
4. Prompt engineering and structured output	**20%**
5. Context management and reliability	**15%**

امتحانی منظرنامہ

Scenario 1: Customer Support Agent

آپ ایک agent جو بناتے ہیں Claude Agent SDK استعمال کرتے ہوئے returns، billing disputes، اور account issues پر مشتمل سنبھالتا ہے agent MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) 80 فیصد+% first-contact resolution مناسب escalation کے ساتھ استعمال کرتا ہے۔

Scenario 2: Claude Code کے ساتھ Code Generation

آپ Claude Code کو development میں استعمال کرتے ہیں تیز کرنے کے لیے: code generation، refactoring، debugging، documentation۔ آپ کو custom slash commands اور CLAUDE.md configuration کے ساتھ integrate کرنا ہے، اور planning mode کے استعمال کو سمجھنا ہے۔

Scenario 3: Multi-Agent Research System

ایک coordinator agent specialized subagents کو tasks سونپتا ہے: web research، document analysis، synthesis، اور report generation۔ سسٹم کو citations کے ساتھ مکمل رپورٹس تیار کرنی چاہئیں۔

Scenario 4: Developer Productivity Tools

یہ agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور routine tasks بنانے، automate کرنے میں مدد دیتا ہے Built-in tools (Read، Write، Bash، Grep، Glob) اور MCP servers استعمال ہوتے ہیں۔

Scenario 5: Claude Code for Continuous Integration

Claude Code کو CI/CD pipeline □میں ضم کریں تاکہ automated code reviews، test generation، اور pull request feedback □ہو□ حاصل Prompts □کو چاہئیں کہ ایسے بننے false positives □ہوں□ کم سے کم

Scenario 6: Structured Data Extraction

□□ معلومات نکالتا دستاویزات سے □سسٹم غیر ساخت□، output کو JSON schemas □ذریعہ □کو validate، □□ کرتا اور high accuracy □اسے □□□ برقرار رکھتا edge cases □چاہئیں□ درست طور پر سنبھالنے

□حصہ I: نظریاتی بنیادیں

یہ حصہ □وہ بنیادی نظریہ بیان کرتا □□ جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت □□□ مواد کو topic کے مطابق — اس سے □ہر domains کے مطابق ترتیب دیا گیا □□، نہ کہ امتحانی concepts اور technologies کی گہری سمجھ بنتی □□□

---

1 باب: Claude API — Model Interaction کی بنیادیں

> دستاویزات: Messages API | Prompt Engineering

1.1 API Request Structure

Claude API ایک request–response model □□□ پر کام کرتا Claude Messages API request □یہ عموماً میں شامل □ہوتا □□:

```
{
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "system": "You are a helpful assistant.",
    "messages": [
        {"role": "user", "content": "Hi!"},
        {"role": "assistant", "content": "Hello!"},
        {"role": "user", "content": "How are you?"}
    ],
    "tools": [...],
    "tool_choice": {"type": "auto"}
}
```

□اہم فیلڈز:

- `model` — model selection (`claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5`)
- `max_tokens` — response □زیادہ سے زیادہ میں tokens
- `system` — system prompt (model کی تعریف □کردار□)
- `messages` — conversation history (**full history □□ بھیجنا ضروری**)

- `tools` — available tools کی definitions
- `tool_choice` — tool selection strategy

1.2 Message Roles

`messages` array میں تین roles ہوتے ہیں:

- `user` — user messages
- `assistant` — model responses (history میں بھیجتے وقت شامل ہوتے ہیں)
- `tool` — tool call results (عملاً `tool_result` content block)

اہم بات: ہر API request میں مکمل conversation history بھیجیں Model requests درمیان کے state محفوظ آزاد ہوتی ہیں call ہیں رکھتا؛ ہر

1.3 `stop_reason`

Claude API response میں `stop_reason` کے جو بتاتا ہے کہ model کیوں رُکا ہے:

Value	Description	Action
`"end_turn"`	Model جواب مکمل کر لیا ہے	user کو نتیجہ دکھائیں
`"tool_use"`	Model tool call چاہتا ہے	tool چلائیں اور result واپس دیں
`"max_tokens"`	Token limit ختم ہو گئی	response truncated؛ limit میں بڑھا سکتے ہیں
`"stop_sequence"`	Stop sequence مل گئی	application logic مطابق کے handle کریں

Agentic systems میں `"tool_use"` اور `"end_turn"` اہم سب سے signals یہی — ہیں loop control کرتے ہیں۔

1.4 System Prompt

System prompt ایک خاص instruction جو ہے context اور behavioral rules یہ ہے متعین کرتا:

- `messages` array الگ؛ ہوتا نہیں حصہ کا `system` field ہے میں دیا جاتا
- user messages ہے پر فوقیت رکھتا
- ایک بار load ہے تا رہتا میں لاگو ہو کر پوری گفتگو
- role، constraints، اور output format ہے تا استعمال ہوتا لیے کے وضاحت کی

امتحان کے لیے اہم: system prompt کی wording tool selection مثال ہے میں کتنی سکتی پر متاثر کر غیر ارادی طور کو ضرورت سے زیادہ `get_customer` کو model کو تصدیق کریں" جیسی ہدایت کے customer طور پر "ہمیشہ استعمال کرنے پر لے جا سکتی ہے

1.5 Context Window

Context window کل متن □و (tokens) □جس □□ model ایک وقت میں process اس میں شامل □□□ کر سکتا □□:

- System prompt
- مکمل message history
- Tool definitions
- Tool results

□اہم مسائل:

1. **Lost-in-the-middle effect:** □لمبا input کی درمیان میں، □ہوتی □ہیں مؤثر □زیادہ معلومات کی آغاز اور اختتام □کہ □و سکتی □ہیں□ حل: □اہم معلومات شروع یا آخر میں رکھیں□ details miss
2. **Tool results □ر □ہونا: کا جمع** tool call context میں output اگر □□□ شامل کرتی tool 40+ fields واپس □ہوں تو □اہم 5 صرف □کر context □□□ ضائع □ہوتا
3. **Progressive summarization:** history compress پر □کرنا numeric values، percentages، اور dates اکثر vague □ہیں □ہو جاتی □ہو

---

2 باب: Tools اور `tool_use`

> دستاویزات: Tool Use

2.1 `tool_use` □□ کیا

`tool_use` □و mechanism □ذریعہ □کہ جس □□ Claude external functions call □□□ کر سکتا Model □□برا واپس کرتا result کر □کہ execute □اسے code بناتا □□؛ آپ کا structured tool call نہیں چلاتا — □و code راست □□□

2.2 Tool Definition

□ر tool JSON schema □ذریعہ □کہ define □□ □ہوتا:

```
{
    "name": "get_customer",
    "description": "Finds a customer by email or ID. Returns the customer profile,
including name, email, order history, and account status. Use this tool BEFORE
lookup_order to verify the customer's identity. Accepts an email (format:
user@domain.com) or a numeric customer_id.",
    "input_schema": {
        "type": "object",
        "properties": {
            "email": {"type": "string", "description": "Customer email"},
            "customer_id": {"type": "integer", "description": "Numeric customer ID"}
        },
        "required": []
    }
}
```

Tool description کے اہم نکات:

1. Description ہی tool selection کا بنیادی mechanism ہے۔ Minimal descriptions غلط selection کا سبب بنتی ہیں۔
2. Description میں یہ شامل کریں: tool کیا واپس کرتا ہے، کیا کرتا ہے، input format کیا ہے، edge cases کیا ہیں، اور یہ کب استعمال ہونا چاہیے۔
3. Overlapping descriptions سے بچیں۔
4. Built-in tools اور MCP tools میں overlap ہو تو MCP tool کو بہتر اور واضح بنائیں۔

2.3 `tool_choice`

Value	Behavior	کب استعمال کریں
`{ "type": "auto" }`	Model خود طے کرتا ہے	default عام
`{ "type": "any" }`	Model کو لازماً tool call کرنی ہوگی	جب structured output چاہیے
`{ "type": "tool", "name": "extract_metadata" }`	خاص tool لازماً call ہوگی	forced first step / order کے لیے

2.4 JSON Schemas for Structured Output

JSON schemas کے ساتھ `tool_use` استعمال کرنا Claude سے structured output کا سب سے reliable طریقہ ہے۔ یہ syntax errors اور کم کرتا ہے required structure، مگر semantic correctness کی ضمانت نہیں دیتا۔

Design اصول:

1. صرف وہ fields required رکھیں جو ہمیشہ ملنے چاہئیں
2. ممکنہ missing data کے لیے nullable fields استعمال کریں
3. `"other"` اور `"unclear"` جیسے enum values شامل کریں تاکہ information ضائع نہ ہو

2.5 Syntax اور Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON، غلط field type	JSON schema ساتھ □کا `tool_use`
Semantic	Totals match □کرتے □نہیں، value غلط field میں	Validation checks، feedback retry

---

3 باب: Claude Agent SDK — Agentic Systems بنانا

> دستاویزات: Agent SDK | Hooks | Subagents | Sessions

3.1 Agentic Loop

Agentic loop میں Claude □بلکہ، صرف جواب □نہیں دیتا actions کی sequence □□ چلاتا:

1. Claude کو tools ساتھ □کا request بھیجیں
2. Response لیں
3. `stop_reason` چیک کریں:
   - `"tool_use"` → tool چلائیں، result history □پھر دوبار، میں ڈالیں request
   - `"end_turn"` → task □نتیجہ، مکمل user کو دیں
4. مکمل □ونے □تک دہرائیں

درست signal: completion □لیے □کا `stop_reason == "end_turn"` □دیکھیں Text parsing یا arbitrary iteration limit □پر □نہیں قابلِ اعتماد

3.2 `AgentDefinition`

```
agent = AgentDefinition(
        name="customer_support",
        description="Handles customer requests for returns and order issues",
        system_prompt="You are a customer support agent...",
        allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
)
```

چار □اہم چیزیں: `name`، `description`، `system_prompt`، اور `allowed_tools` □ Least privilege □اصول اپنائیں

3.3 Coordinator اور Subagents

Multi-agent systems اکثر hub-and-spoke topology □میں استعمال کرتی:

```
                Coordinator
               /     |      \
     Subagent1  Subagent2  Subagent3
```

Coordinator task □□ توڑتا □کو، subagents □□ نتائج جوڑتا □□، کام سونپتا □□، منتخب کرتا □□، errors handle کرتا، اور □□ user تک final □□□ جواب پ□نچاتا

اصول □م□l: subagents کا context □و □□ ہوتا □الگ parent history خود بخود inherit □□ نہ□یں کرتے؛

3.4 `Task` Tool

Subagents `Task` tool □ذریعہ □کا spawn □یں□ □ہوتے □ہیں Coordinator □کا `allowed_tools` میں `"Task"` □ہونا چا□ئیے□□

صحیح context passing:

- `Task: "Analyze the document"` کافی نہ□یں
- `Task: "Analyze this document. Full text: ... Output schema: ..."` □□ درست

Coordinator ایک response میں multiple `Task`s □□□لہذا، □□ بھیج سکتا parallel subagents □یں□ ممکن

3.5 Hooks

Hooks lifecycle مخصوص □کے points پر work □یں□ کرتے □

PostToolUse tool result کو model □لا□ سے پ□نچانے تک normalize □□ کر سکتا:

```
@hook("PostToolUse")
def normalize_dates(tool_result):
        return normalized_result
```

PreToolUse policy violation block □□ کر سکتا:

```
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
        if tool_call.name == "process_refund" and tool_call.args.amount > 500:
                return redirect_to_escalation(tool_call)
```

یاد رکھیں: hooks deterministic □یں□ □ہوتے □یں؛ prompt instructions probabilistic □یں□ ہوتی□ Critical business rules □لیے □کے hooks □یں□ بہتر

---

4 باب: Model Context Protocol (MCP)

> دستاویزات: MCP | Tools | Resources | Servers

4.1 MCP □□ کیا

MCP ایک open protocol جو □□ external systems کو Claude سے جوڑتا □□□ اس کے تین بنیادی resource types ہیں□:

1. **Tools** — actions کے لیے functions
2. **Resources** — context data (documentation، schemas، catalogs)
3. **Prompts** — predefined task templates

4.2 MCP Servers

MCP server ایک process جو □□ tools/resources expose □□□ کرتا Connect پر ہونے tools discover ہو جاتے ہیں اور descriptions ہیں ہوتے مطابق استعمال کے

4.3 Configuration

Project config (`.mcp.json`) ٹیم کے لیے:

```
{
    "mcpServers": {
        "github": {
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-github"],
            "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
        }
    }
}
```

User config سے آتی ہیں environment variables secrets میں ہوتی □□؛ Project config version control (`~/.claude.json`) personal/experimental servers □□□ ہوتی لیے کے

4.4 `isError`

MCP tool errors میں `isError: true` استعمال کریں□ Generic errors بجائے structured error واپس کریں:

```
{
    "isError": true,
    "content": {
        "errorCategory": "transient",
        "isRetryable": true,
        "message": "The service is temporarily unavailable."
    }
}
```

4.5 Resources

Resources □و data جو □یں agent بغیر action □□ پڑھ سکتا □کیا: schemas، docs، content catalogs، summaries□یا □ exploratory tool calls □یں□ □کم کرتا

---

5 باب: Claude Code — Configuration اور Workflows

> دستاویزات: Claude Code | Memory / CLAUDE.md | Skills | MCP | Hooks | Sub-agents | GitHub Actions | Headless

5.1 CLAUDE.md Hierarchy

CLAUDE.md □□ سکتا □و پر سطحوں تین:

- **User-level:** `~/.claude/CLAUDE.md` — صرف اسی user پر لاگو
- **Project-level:** `.claude/CLAUDE.md` یا root `CLAUDE.md` — پوری team □لیا □کا
- **Directory-level:** subdirectories میں CLAUDE.md — مخصوص folder conventions

5.2 `@path` Syntax

CLAUDE.md میں `@path` □سے external files refer □تاکہ □یں □کیا جا سکتا □کیا config modular □و:

```
Coding standards @./standards/coding-style.md میں ہیں
Test rules @./standards/testing-requirements.md میں ہیں
```

5.3 `.claude/rules/`

`.claude/rules/` rules کو topic □سے لحاظ □کے organize اور □□ کرتا `paths` □ذریعے □کے conditional loading □□ دیتا:

```
---
paths: ["src/api/**/*"]
---

API files ہیں کہ لیے async/await اور explicit error handling استعمال کریں
```

5.4 Commands اور Skills

`.claude/commands/` project commands □جبکہ، □□ لیے □کے `~/.claude/commands/` personal commands □□لیے □کے `.claude/skills/` میں frontmatter ساتھ □کے skills □یں□ ملتی

5.5 Skills اور `context: fork`

`context: fork` skill کو isolated context □□□ میں چلاتا `allowed-tools` □سے tool access اور، محدود کریں `argument-hint` □سے required input □مانگیں

5.6 Planning Mode

Planning mode بڑی architectural تبدیلیوں، multiple approaches، اور unfamiliar codebases □ترین □لیے □کے □□□ Direct execution اور واضح □چھوڑ changes □□□ تر □لیے □کے

5.7 `/compact` اور `/memory`

`/compact` context compress اور، □□ کرتا `/memory` notes اور preferences □□□ سنبھالنے میں مدد دیتا

5.8 Claude Code CLI for CI/CD

`-p` / `--print` non-interactive mode □□ تو□ استعمال □لیے □کے:

```
claude -p "Analyze this pull request for security issues"
```

Structured output □لیے □کے `--output-format json` اور `--json-schema` □استعمال کریں

---

6 باب: Prompt Engineering — Advanced Techniques

> دستاویزات: Prompt Engineering | Anthropic Cookbook

6.1 Few-shot Prompting

Few-shot prompting 4–2 میں examples کر □د expected behavior □یہ □□□ دکھایا جاتا vague instructions □سے □کیونکہ □□ مؤثر □زیاد example ambiguity □□□ کم کرتا

6.2 Explicit Criteria

Vague instructions □بجائ□ کا clear criteria کیا دیں: کیا flag کیا، □□ کرنا ignore اور، □□ کرنا severity □کیس decide □□□ کرنی

6.3 Prompt Chaining

Complex task کو focused steps میں توڑیں: local analysis، پھر integration pass□یا □ attention dilution کم کرتا □□□

6.4 Interview Pattern

Claude □سا clarifying questions پوچھوانا design ambiguity جب خاص طور پر، □□ لاتا □سامنا domain unfamiliar □ہو

6.5 Validation and Retry

Extraction fail □ہو تو original document، incorrect output، اور specific validation error ساتھ □کا retry □کریں

6.6 Self-correction

Model stated value اور calculated value دونوں نکال□؛ conflict □ہو تو discrepancy handle □کریں

---

7 باب: Message Batches API

> دستاویزات: Message Batches

7.1 Overview

Message Batches API asynchronous processing 50 :□□ □لیا □کا% savings، up to 24 hours window، اور multi-turn tool calling supported □نہیں؛ `custom_id` □سا requests اور responses link □ہیں □ہوتا

7.2 کب استعمال کریں

Pre-merge checks اور interactive tasks □لیا □کا synchronous API، □جبکہ overnight reports اور bulk jobs □کا □لیا Batch API □استعمال کریں

7.3 `custom_id`

`custom_id` result کو original document □سے جوڑنا□، failures identify کرنا□، اور صرف failed items □دوبارہ submit □□□ کرنا□ میں مدد دیتا

باب 8: Task Decomposition Strategies

8.1 Fixed Pipelines

Predictable tasks □لیے□ fixed pipeline □جیسے□، استعمال کریں Document → Extraction → Validation → Final output□

8.2 Dynamic Decomposition

Open-ended tasks میں subtasks intermediate results □پر□ کی بنیاد پر بنتے□ □ہیں□

8.3 Multi-pass Code Review

Large PRs □لیے□ per-file analysis کے بعد cross-file integration pass □کریں تاکہ attention dilution □کو□ کم

باب 9: Escalation اور Human-in-the-Loop

9.1 کب Escalate کریں

Explicit user request، policy gaps، progress □ہونا□ □نہ□، threshold اوپر □سے financial action، یا ambiguous matching کی صورت میں escalate □کریں□ Sentiment analysis اور self-rated confidence unreliable □ہیں□

9.2 Escalation Patterns

□مسئلہ□ □پہلے□ resolve کرنا□ کی کوشش کریں، پھر اگر policy gap یا explicit insistence □ہو تو□ human کو hand off □کریں□

9.3 Structured Handoff

Escalation پر customer ID، issue summary، actions taken، اور recommended action □شامل کریں تاکہ human operator کو مکمل context □□ملے□

9.4 Confidence Calibration

Field-level confidence scores، calibration on labeled data، اور stratified sampling استعمال کریں□

---

10 باب: Multi-agent Systems میں Error Handling

10.1 Error Categories

Transient errors retryable ہیں□، validation errors input fix مانگتی ہیں□، business errors policy-driven ہوتی□ ہیں، اور permission errors generally escalate کی جاتی ہیں□

10.2 Anti-patterns

Generic “search unavailable” response، silent suppression، یا پورا workflow abort کرنا نقصان دہ □□□ Structured errors واپس کریں□

10.3 Structured Subagent Error

Structured error میں failure type، attempted query، partial results، alternative approaches، اور coverage impact شامل کریں□

10.4 Final Synthesis میں Coverage

Report میں واضح کریں کہ کون سی sections fully covered ہیں□، کون سی partially covered ہیں□، اور کہاں gaps باقی ہیں□

---

11 باب: Production Systems میں Context Management

11.1 Facts Separate Block میں

Transactional facts کو persistent case facts block میں رکھیں تاکہ□ summarization باوجود کہ□ critical data ضائع نہ□ ہو□

11.2 Tool Results Trimming

PostToolUse hook صرف سہ□ relevant fields رکھیں□

11.3 Position-aware Input

Important findings اور شروع میں action items □تاکہ آخر میں رکھیں lost-in-the-middle effect □و□ کم

11.4 Scratchpad Files

Verbose findings scratchpad □تاکہ میں لکھیں long investigations میں context □□□ر محفوظ

11.5 Subagents کو Delegate کرنا

Exploration subagents اور دیں میں main context میں صرف summary □رکھیں

11.6 Structured State Persistence

□ر agent اپنی state manifest یا JSON file میں export □تاکہ □کر crash recovery □و□ ممکن

---

12 باب: Provenance کو محفوظ رکھنا

12.1 Attribution Loss

Claim ساتھ □کہ source URL، publication date، اور confidence □رکھیں

12.2 Conflicting Data

اگر sources مختلف values کو دونوں تو دیں attribution اور رکھیں محفوظ ساتھ □کہ conflict flag □کریں

12.3 Dates

Temporal differences کو contradiction □ا□ پ□ سے □سمجھنے publication dates □دیکھیں

12.4 Content Type مطابق □کہ Render

Financial data tables میں، news prose میں، technical findings structured lists اور، میں time series chronological order □میں دکھائیں

---

13 باب: Claude Code □ کا Built-in Tools

13.1 Tool Selection Reference

Task	Tool	Example
pattern □سے file تلاش کرنا	**Glob**	`**/*.test.tsx`
file contents میں search	**Grep**	function name, error message
file پڑھنا	**Read**	analysis □لیے □کے
نیا file بنانا	**Write**	scratch □سے
precise edit	**Edit**	unique text match
shell command	**Bash**	git، npm، tests

13.2 Incremental Investigation

ایک ساتھ سب files □لا□؛ پ□ڑھیں □نہ entry points، پھر usages، پھر consumer files □دیکھیں

13.3 Read + Write Fallback

اگر Edit fail □ہو تو Read کریں، تبدیلی programmatically کریں، پھر Write □کریں

□حصہ II: امتحانی ڈومین نوٹس

---

1 ڈومین: Agent Architecture اور Orchestration (27%)

Agentic loops، coordinator–subagent architecture، `Task` □ذریعے □کے spawning، hooks، escalation، اور multi-step workflows □پر فوکس کریں

2 ڈومین: Tool Design اور MCP Integration (18%)

Tool descriptions، `tool_choice`، JSON schema output، structured errors، MCP servers، resources، اور built-in tools □کے فرق کو سمجھیں

3 ڈومین: Claude Code Configuration اور Workflows (20%)

CLAUDE.md hierarchy، `.claude/rules/`، commands، skills، planning mode، `context: fork`، اور CI/CD integration □یں□ م□ا

4 ڈومین: Prompt Engineering اور Structured Output (20%)

Few-shot prompting، explicit criteria، prompt chaining، validation/retry loops، JSON schema enforcement، اور batch-friendly prompting □دیں □توج پر

5 ڈومین: Context Management اور Reliability (15%)

Summarization risks، lost-in-the-middle، trimming tool output، confidence calibration، escalation، اور provenance preservation □یاد رکھیں

عملی مشقیں

Exercise 1: Multi-tool Agent with Escalation Logic

Goal: tool integration، structured errors، اور escalation ساتھ □کا agent loop □بنائیں

Exercise 2: Configuring Claude Code for Team Development

Goal: CLAUDE.md، custom commands، path-specific rules، اور MCP servers configure □کریں

Exercise 3: Structured Data Extraction Pipeline

Goal: JSON schema، validation/retry loops، اور Message Batches API ساتھ □کا extraction pipeline □بنائیں

Exercise 4: Designing and Debugging a Multi-agent Research Pipeline

Goal: subagent orchestration، explicit context passing، error propagation، اور source tracking implement □کریں

Appendix: Technologies and Concepts

Technology	Key aspects

Claude Agent SDK	agent loops, `stop_reason`, hooks, `Task`, `allowedTools`
MCP	servers, tools, resources, `isError`, `.mcp.json`
Claude Code	CLAUDE.md, `.claude/rules/`, `.claude/commands/`, `.claude/skills/`, planning mode
Claude Code CLI	`-p`, `--output-format json`, `--json-schema`
Claude API	`tool_use`, `tool_choice`, `stop_reason`, `max_tokens`
Message Batches API	50% savings, 24-hour window, `custom_id`
JSON Schema	required/optional, nullable, enum values
Few-shot prompting	ambiguous scenarios □لیے □کے examples
Prompt chaining	sequential decomposition
Context window	token budget, lost-in-the-middle
Confidence calibration	field-level scoring, stratified sampling

---

Out-of-Scope Topics

□یں موضوعات امتحان میں شامل نہ□یں □یں:

- Claude models کی fine-tuning
- Authentication، billing، یا account management
- مخصوص programming languages کی تفصیلی implementation
- MCP servers کی hosting/infrastructure
- Claude کی internal architecture یا model weights
- Constitutional AI، RLHF، یا safety training
- Embeddings یا vector databases کی implementation details
- Computer use / browser automation
- Vision / image analysis
- Streaming API یا SSE
- Rate limiting اور detailed API cost calculations
- OAuth اور API key rotation details
- Cloud-provider-specific configs
- Performance benchmarks
- Prompt caching implementation details
- Token counting algorithms

Preparation Recommendations

1. Claude Agent SDK ایک ساتھ کا agent بنائیں
2. Claude Code کو real project پر configure کریں
3. MCP tools design اور test کریں
4. Data extraction pipeline بنائیں
5. Prompt engineering practice کریں
6. Context management patterns سیکھیں
7. Escalation اور human-in-the-loop workflows سمجھیں
8. Practice exam حل کریں